Reinforcement Learning vs. Stochastic Control Approaches to Optimal Market Making: A Controlled Simulation Study

Cem Vural
Boğaziçi University - Master's in Financial Engineering
Advisor: Mehmet Yasin Ulukuş

Abstract:

Market making involves continuously providing liquidity by quoting bid and ask limit orders while managing inventory risk under uncertain price dynamics. Classical approaches, most notably the Avellaneda–Stoikov model, formulate this problem as a stochastic control problem and yield closed-form optimal quoting strategies under restrictive assumptions. While analytically tractable, these models struggle to accommodate realistic market features such as jumps, volatility regime changes, and partial observability.

This graduation project conducts a controlled simulation study comparing stochastic control–based market-making strategies with reinforcement learning (RL) approaches. A unified environment is developed in which all agents—analytical, heuristic, and learning-based—operate under identical execution rules, state representations, and inventory constraints. Market environments are progressively enriched to relax classical assumptions, allowing a systematic evaluation of robustness as market complexity increases.

Performance is assessed using risk-adjusted profitability metrics and inventory exposure across repeated trading episodes. The results show that analytical strategies remain effective in idealized settings, while heuristic baseline agents and reinforcement learning agents exhibit relatively better adaptability in more complex and non-stationary environments. These findings help identify the conditions under which learning-based market making could provide advantages over classical stochastic control models. Further research should focus on the models' sensitivity to environment hyperparameters such as volatility and order execution intensity as well as modeling with higher dimensional state representations to capture more of the complexity of real world limit-order books (LOBs).

https://github.com/cemvural00/market_making_rl/

Acknowledgements

I would like to acknowledge the use of AI-assisted tools, including ChatGPT, Cursor, and Gemini, during the development of both the codebase and the written report for this project. These tools were used as supportive aids for tasks such as code debugging, generation, refactoring, documentation drafting, and language refinement. All methodological choices, implementations, experiments, analyses, and final interpretations remain my own responsibility. The use of these tools did not replace independent reasoning but served to enhance productivity and clarity throughout the research and writing process.

Contents

1. Introduction
2. Literature Review
2.1 Theoretical Foundations: Stochastic Control and Inventory Risk
2.2 Data-Driven Approaches: Reinforcement Learning
2.3 Benchmarking and the Reality Gap
2.4 Summary and Research Gap
3. Methodology
3.1 Environment Design
3.1.1 General Framework
3.1.2 Price Dynamics
3.1.3 Order Arrival and Execution Model
3.1.4 Inventory and Cash Dynamics
3.1.5 State Representation
3.1.6 Action Space
3.1.7 Reward Function
3.1.8 Design Rationale
3.2 Agents
3.2.1 Unified Agent Interface (Abstract Agent Class)
3.2.2 Reinforcement Learning Agents
3.2.2.1 Proximal Policy Optimization (PPO) Agent
3.2.2.2 Deep Proximal Policy Optimization (Deep PPO) Agent
3.2.2.3 Recurrent PPO Agent with LSTM Memory
3.2.2.4 Soft Actor–Critic (SAC) Agent
3.2.2.5 Deep SAC Agent (High-Capacity Approximation)
3.2.2.6 Twin Delayed Deep Deterministic Policy Gradient (TD3) Agent
3.2.3 Analytic Agents (Stochastic Control–Based Agents)
3.2.3.1 AS-Inspired Simple Heuristic Agent
3.2.3.2 Avellaneda–Stoikov Closed-Form Agent
3.2.4 Heuristic Baseline Agents
3.2.4.1 Simple Heuristic Agents
3.2.4.1.1 Fixed-Spread Agent
3.2.4.1.2 Market-Order-Only Agent
3.2.4.2 Inventory-Based Heuristic Agents
3.2.4.2.1 Inventory-Based Spread Scaling Agent
3.2.4.2.2 Inventory-Based Quote Shifting Agent
3.2.4.3 Microstructure-Based Heuristic Agents

3.2.4.3.1 Mid-Price Following Agent
3.2.4.3.2 Last-Look (Momentum-Reactive) Agent
3.2.4.4 Random and Zero-Intelligence Agents
3.2.4.4.1 Zero-Intelligence Agent (Uniform)
3.2.4.4.2 Noise Trader (Gaussian)
3.2.4.4.3 Noise Trader (Uniform-Band)
3.3 Experimental Design and Experiment Matrix
3.3.1 General Evaluation Protocol
3.3.2 Training and Evaluation Separation
3.3.3 Performance Metrics
3.3.4 Experiment Matrix
3.3.5 Reproducibility and Result Storage
4. Results (See Appendix: E for the comprehensive results data)
4.1 Cross-Category Summary
4.2 Profitability by Environment
4.2.1 ABM family (closest to AS assumptions, then enriched with jumps/regimes)
4.2.2 GBM family (scale effects and multiplicative dynamics)
4.2.3 OU family (mean reversion + jump/regime enrichments)
4.3 Risk-adjusted performance and downside risk
4.3.1 Sharpe ratios: analytical strategies are more reliable
4.3.2 Tail risk: RL underperforms in VaR/ES almost everywhere
4.4 Inventory outcomes
4.5 Cross-Environment Performance Insights
4.5.1 Robustness across environments (win-rates by category)
5. Conclusion
5.1 Core findings
5.2 Why did it play out this way?
5.2.1 Market dynamics and misspecification channels
5.2.2 Execution model effects (Poisson intensity with exponential decay)
5.2.3 RL stability and why off-policy methods did better
5.3 What could enrich follow-up studies?
5.3.1 Hyperparameter and environment sensitivity analysis
5.3.2 Higher dimensional state representations (closing the information gap)
5.3.3 More advanced simulation and real LOB data
5.3.4 Agent design extensions: reward functions and action spaces
5.4 Final interpretation

6. References
APPENDIX: A — The Avellaneda–Stoikov Optimal Market Making Model
A.1 Introduction
A.2 Model Assumptions
A.3 The Hamilton–Jacobi–Bellman (HJB) Equation
A.4 Reservation Price
A.5 Optimal Spread
A.6 Optimal Bid and Ask Quotes
A.7 Connection to Project Implementation
A.8 Summary Table of Formulas
A.9 Why the AS Model Is Foundational in Market Making
APPENDIX: B — Reinforcement Learning Formulation of the Market-Making Problem
B.1 Markov Decision Process Formulation
B.2 State Space
B.3 Action Space and Quote Parameterization
B.4 Transition Dynamics
B.5 Reward Function
B.6 Policy Classes and Objective
B.7 Relation to Stochastic Control
APPENDIX: C — Environment Specifications and Parameterization
C.1 General Environment Structure
C.2 Arithmetic Brownian Motion (ABM) Environments
C.2.1 ABM Vanilla
C.2.2 ABM with Jump Diffusion
C.2.3 ABM with Regime-Switching Volatility
C.2.4 ABM with Jumps and Regime Switching
C.3 Geometric Brownian Motion (GBM) Environments
C.4 Ornstein–Uhlenbeck (OU) Environments
C.5 Execution and Inventory Parameters
C.6 Reproducibility via Configuration Presets
C.7 Parameter Configuration
APPENDIX: D — Agent Specifications and Hyperparameter Configurations
D.1 Reinforcement Learning Agents
D.1.1 Proximal Policy Optimization (PPO)
D.1.2 Recurrent PPO (LSTM-PPO)
D.1.3 Soft Actor–Critic (SAC)

D.1.4 Deep SAC (LSTM-SAC Approximation)
D.1.5 Twin Delayed DDPG (TD3)
D.2 Analytical Stochastic Control Agents
D.2.1 Avellaneda–Stoikov Closed-Form Agent
D.2.2 AS-Inspired Heuristic Agent
D.3 Heuristic Baseline Agents
D.3.1 Fixed-Spread Agents
D.3.2 Inventory-Based Heuristics
D.3.3 Trend and Microstructure-Based Heuristics
D.3.4 Market Order and Noise-Based Agents
D.4 Configuration Management and Reproducibility
APPENDIX: E — Results and Tables

1. Introduction

Market making plays a central role in financial markets by providing liquidity, reducing transaction costs, and facilitating continuous price discovery. A market maker simultaneously quotes buy and sell limit orders and profits from the bid–ask spread while managing the risk arising from inventory accumulation and adverse price movements. The design of optimal market-making strategies is therefore a fundamental problem of quantitative finance.

A seminal contribution to this problem is the model introduced by Avellaneda and Stoikov (2008), which formulates market making as a continuous-time stochastic control problem. Under stylized assumptions on price dynamics and order arrivals, the model yields closed-form expressions for optimal bid and ask quotes that explicitly balance expected profits from liquidity provision against inventory risk. Due to its analytical tractability and economic interpretability, the Avellaneda–Stoikov framework has become a standard benchmark in both academic research and industrial applications. (See Appendix: A)

Despite its elegance, the Avellaneda–Stoikov model relies on restrictive assumptions. It assumes the market mid-price dynamics follow a driftless Brownian Motion, symmetric and stationary order arrival intensities modelled by Poisson Processes, and complete observability of the state variables. Real-world markets, however, exhibit features such as jumps, volatility clustering, regime changes, and partial observability, which are difficult to incorporate into closed-form solutions. These limitations motivate the exploration of data-driven and adaptive approaches that can operate effectively under more complex and realistic market conditions.

Recent advances in reinforcement learning (RL) have made it possible to address sequential decision-making problems in high-dimensional and stochastic environments. In the context of market making, reinforcement learning offers a flexible framework in which quoting policies can be learned directly through interaction with a simulated or real market environment, without requiring strong parametric assumptions. Several recent studies (2018, 2021, 2022) have demonstrated that RL agents can approximate or even outperform traditional analytical strategies in settings where the assumptions of classical models break down.

This report compares the performance of classical optimal market-making models and modern reinforcement learning approaches. The Avellaneda–Stoikov model is treated as a principled analytical baseline, against which a family of reinforcement learning agents, and a number of heuristic baseline approaches are compared. These agents range from standard policy-gradient methods to deep and recurrent architectures capable of capturing nonlinearities and temporal dependencies. All agents are evaluated within a unified simulation framework that allows systematic control over price dynamics, execution processes, and market regimes.

The analysis proceeds in stages. First, reinforcement learning agents are trained and evaluated in synthetic environments that replicate the assumptions of the Avellaneda–Stoikov model, enabling a direct comparison between analytical and learned strategies. The environment is then progressively enriched to include features such as jump diffusion, mean reversion, and regime-switching volatility. This progression allows an explicit examination of how each modeling approach responds as the market departs from idealized assumptions. Finally, the implications of

these results for real-world market making and possible future extensions to historical limit order book data are discussed.

By combining closed-form stochastic control models with reinforcement learning, this work aims to clarify the conditions under which classical market-making strategies remain robust, and to identify scenarios in which learning-based methods offer meaningful advantages. The contribution of this study is not to replace analytical models, but to position them within a broader methodological framework that integrates economic theory with modern machine learning techniques.

2. Literature Review

The problem of optimal market making has historically been framed as an inventory management problem within the field of stochastic optimal control. However, the rigid assumptions required to derive closed-form solutions in these frameworks have increasingly led researchers to explore model-free Reinforcement Learning (RL) approaches. This chapter reviews the evolution of market-making strategies from foundational analytical models to modern deep learning implementations, highlighting the specific limitations of classical theory and how the newer approaches are trying to solve it.

2.1 Theoretical Foundations: Stochastic Control and Inventory Risk

The economic intuition for inventory-based market making was first formalized by Ho and Stoll (1981; 1983), who modeled the dealer's problem as maximizing the expected utility of terminal wealth. They demonstrated that a risk-averse dealer faces a trade-off between profit maximization from the bid-ask spread and the "inventory risk" associated with holding a non-zero position. Their seminal finding was that optimal quotes should be centered not around the market mid-price, but around a "reservation price" that adjusts dynamically to induce mean reversion in the inventory level (Ho & Stoll, 1981).

Building on this foundation, Avellaneda and Stoikov (2008) provided a continuous-time formulation of the problem. By modeling the mid-price as a driftless Brownian Motion and assuming that the arrival rates of limit orders decay exponentially with the distance from the mid-price (the "intensity function"), they derived a closed-form approximation for optimal quotes. Specifically, under the assumption of exponential utility, the Avellaneda-Stoikov model prescribes a reservation price $r(s, q, t)$ and a spread $\delta(t)$ that are explicit functions of the current inventory q, market volatility σ and the market maker's risk aversion parameter γ (Avellaneda & Stoikov, 2008).

While the Avellaneda-Stoikov framework serves as the standard benchmark in the literature, its reliance on specific parametric forms limits its applicability to complex markets. Subsequent research has focused on relaxing these constraints. Guéant, Lehalle, and Fernandez-Tapia (2013) and Guéant (2017) generalized the framework to accommodate a wider class of intensity functions. Crucially, Guéant (2017) demonstrated that the Hamilton-Jacobi-Bellman (HJB) equations governing the optimal control problem could often be reduced to systems of Ordinary Differential Equations (ODEs), preserving analytical tractability even under more general assumptions about market liquidity.

Further extensions have attempted to incorporate stylized facts of asset returns. For instance, Aydoğan, Uğur, and Aksoy (2022) extended the classical framework to include Heston stochastic volatility and jump processes. Their work highlights that while analytical models can theoretically handle features like volatility clustering, the resulting systems of partial differential equations become computationally prohibitive to solve in real-time, necessitating numerical approximations that erode the primary advantage of the closed-form approach (Aydoğan et al., 2022).

2.2 Data-Driven Approaches: Reinforcement Learning

The rigidity of stochastic control models—specifically their inability to adapt to changing market regimes or non-Markovian dynamics—has motivated the application of Reinforcement Learning (RL). In an RL framework, the market maker is modeled as an agent that learns a policy $\pi(s)$ mapping market states to quoting actions to maximize a cumulative reward signal, typically a risk-adjusted version of Profit and Loss (PnL).

Early applications, such as Chan and Shelton (2001) and Kim et al. (2002), utilized tabular Q-learning methods. However, these approaches struggled with the "curse of dimensionality" inherent in Limit Order Book (LOB) data. Modern research has therefore shifted toward Deep Reinforcement Learning (DRL), which uses neural networks to approximate value functions across continuous high-dimensional state spaces.

A key challenge in applying DRL to market making is the design of a stable reward function. Spooner et al. (2018) addressed this by introducing an "Asymmetrically Dampened PnL" reward function. Working with high-fidelity LOB simulations, they demonstrated that standard PnL rewards lead to high-variance learning updates, whereas their dampened metric allowed the agent to converge to stable market-making policies that outperformed benchmark strategies. Their work emphasizes that successful RL applications require careful engineering of the reward signal to reflect the economic realities of inventory risk (Spooner et al., 2018).

Beyond single-agent optimization, RL has proven effective in modeling strategic interactions. Ganesh et al. (2019) utilized a multi-agent reinforcement learning (MARL) framework to simulate a dealer market. Their agents, trained via policy gradient methods, autonomously learned sophisticated behaviors such as "skewing" quotes—adjusting prices asymmetrically to internalize order flow based on inventory imbalances and competitor actions. Importantly, Ganesh et al. (2019) found that these learned behaviors aligned with the theoretical predictions of Ho and Stoll (1983) but were achieved without any prior knowledge of the market model, validating RL's capability to rediscover economic principles from raw experience.

2.3 Benchmarking and the Reality Gap

Despite the reported successes of RL, a significant gap remains between simulation results and real-world applicability. A comprehensive survey by Gašperov et al. (2021) notes that many RL studies rely on simplified market simulators that fail to capture the complex feedback loops of real electronic markets. They argue that while RL agents often achieve higher Sharpe ratios than analytical baselines in backtests, their performance is highly sensitive to the "simulation-to-reality" gap, particularly regarding execution latency and market impact (Gašperov et al., 2021).

Recent comparative studies have attempted to rigorously quantify this performance differential. Carlsson and Regnell (2022) systematically compared tabular Q-learning and Double Deep Q-Networks (DDQN) against the Avellaneda-Stoikov baseline. Their analysis revealed that while Deep RL agents excelled in complex, non-linear environments, they were prone to overfitting and instability in simpler settings where tabular methods or analytical models were sufficient. Furthermore, they highlighted the critical importance of state representation, finding that including order book depth and imbalance features significantly improved the agent's ability to anticipate short-term price movements (Carlsson & Regnell, 2022).

2.4 Summary and Research Gap

The literature establishes two distinct paradigms: the rigorous but rigid world of stochastic control (Ho & Stoll, 1983; Avellaneda & Stoikov, 2008; Guéant, 2017) and the flexible but opaque world of Reinforcement Learning (Spooner et al., 2018; Ganesh et al., 2019). While studies like Aydoğan et al. (2022) have pushed the boundaries of the former and Gašperov et al. (2021) have surveyed the latter, there remains a lack of unified comparative analysis that systematically introduces market complexities—such as jumps and regime switches—to test the breaking points of both approaches. This project aims to fill this gap by benchmarking the Avellaneda-Stoikov model against RL agents in a progressively complex simulation environment, thereby isolating the specific conditions under which learning-based strategies offer a tangible advantage over closed-form solutions.

3. Methodology

This section describes the simulation framework used to evaluate and compare analytical and reinforcement learning–based market-making strategies. The focus of this section is the environment and agent design, which constitutes the experimental backbone of the study. All agents—analytical, heuristic, and learning-based—are trained and evaluated within the same family of environments to ensure fair and controlled comparisons.

3.1 Environment Design

3.1.1 General Framework

All environments are implemented as discrete-time episodic simulations of a single-asset limit order market. Each episode represents a finite trading horizon T, which can be interpreted as a trading day. Time is discretized into steps of length Δt (set to 0.0001, leading to 10,000 time steps per episode, by default), and at each step the market maker posts bid and ask quotes around the current mid-price.

The environment explicitly models three interacting components:

1. **Mid-price dynamics**, which govern the evolution of the reference price.
2. **Order arrival and execution**, which determine whether posted quotes are filled.
3. **Inventory and cash dynamics**, which determine the agent’s profit and risk exposure.

This structure closely follows the modeling assumptions of classical optimal market-making theory, while remaining flexible enough to support richer dynamics such as jumps, mean reversion, and Markov volatility regime changes.

3.1.2 Price Dynamics

The mid-price S_t evolves exogenously according to a specified stochastic process. Several classes of price dynamics are considered to progressively relax the assumptions of the classical Avellaneda–Stoikov model.

Arithmetic Brownian Motion (ABM)

The baseline environment assumes driftless arithmetic Brownian motion:

$$dS_t = \sigma\, dW_t,$$

where σ denotes volatility and W_t is a standard Brownian motion. This specification matches the assumptions of the original Avellaneda–Stoikov model and serves as the primary benchmark environment.

In discrete time, the price update is implemented as:

$$S_{t+\Delta t} = S_t + \sigma\sqrt{\Delta t}\, \varepsilon_t,\ \varepsilon_t \sim \mathcal{N}(0,1).$$

Geometric Brownian Motion (GBM)

To account for proportional price changes, a geometric Brownian motion variant is also considered:

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t.$$

This specification is relevant for longer horizons and assets where relative price changes are more stable than absolute changes. In the baseline experiments, the drift μ is always set to zero to avoid introducing directional bets.

Ornstein–Uhlenbeck (OU) Mean Reversion

To capture mean-reverting price behavior, the mid-price is modeled as an Ornstein–Uhlenbeck process:

$$dS_t = \kappa(\mu - S_t)dt + \sigma dW_t,$$

where κ controls the speed of mean reversion and μ is the long-run mean (set to S_0 by default). This environment tests whether agents can exploit predictable price dynamics while managing inventory risk.

Jump Diffusion Extensions

To model discontinuous price movements, jump diffusion terms are introduced:

$$dS_t = \sigma dW_t + J_t dN_t,$$

where N_t is a Poisson process with intensity λ_J, and jump sizes J_t are drawn from a specified distribution. This extension captures sudden price moves due to news or liquidity shocks and represents a significant departure from the assumptions of closed-form market-making models.

Regime-Switching Volatility

In regime-switching environments, volatility alternates between two latent states (low and high volatility regimes). The regime follows a discrete-time Markov chain with a fixed transition matrix, and the volatility parameter σ depends on the current regime.

This design introduces non-stationarity and partial observability, motivating the use of recurrent reinforcement learning agents.

3.1.3 Order Arrival and Execution Model

Order executions are modeled using independent Poisson processes for the bid and ask sides. The arrival intensity depends exponentially on the distance between the quoted price and the mid-price:

$$\lambda^{\text{bid}}(\delta_b) = Ae^{-k\delta_b},\ \lambda^{\text{ask}}(\delta_a) = Ae^{-k\delta_a},$$

where:

- $\delta_b = S_t - p_t^{\text{bid}}$,
- $\delta_a = p_t^{\text{ask}} - S_t$,
- A controls baseline market activity,
- k controls the sensitivity of execution probability to quote distance.

In each time step, executions are approximated using Bernoulli trials with probabilities $\lambda\Delta t$, which is a standard discretization of Poisson arrivals for small Δt.

This execution model directly mirrors the assumptions of the Avellaneda–Stoikov framework and provides a clean, interpretable link between quoting behavior and fill probabilities.

3.1.4 Inventory and Cash Dynamics

The market maker maintains:

- inventory q_t,
- cash position X_t.

Upon a bid fill:

$$q_{t+1} = q_t + 1,\ X_{t+1} = X_t - p_t^{\text{bid}}.$$

Upon an ask fill:

$$q_{t+1} = q_t - 1,\ X_{t+1} = X_t + p_t^{\text{ask}}.$$

Terminal wealth is defined as:

$$W_T = X_T + q_T S_T.$$

Inventory is constrained to lie within fixed bounds to avoid unrealistic leverage (max q set to 40 by default).

3.1.5 State Representation

At each time step, the agent observes a low-dimensional state vector designed to balance informativeness and tractability:

$$\text{state}_t = [\frac{t}{T},\ \frac{S_t}{S_0},\ S_t - S_{t-\Delta t},\ \frac{q_t}{q_{\max}}].$$

This representation includes:

- normalized time remaining,
- normalized mid-price,
- recent price change,
- normalized inventory.

Normalization ensures numerical stability and allows agents to generalize across assets and price levels. The state intentionally excludes full order book information in order to remain comparable with classical analytical models.

3.1.6 Action Space

Agents do not quote absolute prices directly. Instead, they output continuous control variables in a bounded action space:

$$a_t = (\text{spread_factor}, \text{skew_factor}) \in [-1,1]^2.$$

These are mapped to actual quotes via:

- a base half-spread scaled by the spread factor,
- an inventory-dependent skew that shifts both bid and ask prices.

This parametrization enforces realistic quoting behavior, stabilizes learning, and allows direct comparison with analytical solutions.

3.1.7 Reward Function

The per-step reward is defined as the change in marked-to-market profit minus an inventory risk penalty:

$$r_t = (W_{t+1} - W_t) - \eta q_{t+1}^2 \Delta t,$$

where η controls inventory aversion.

This reward structure approximates the objective of maximizing expected terminal wealth under inventory risk and aligns closely with the exponential utility maximization underlying the Avellaneda–Stoikov model.

3.1.8 Design Rationale

The environment design deliberately prioritizes:

- analytical consistency with classical models,
- modular extensibility to richer dynamics,
- computational efficiency for large-scale simulation,
- reproducibility and interpretability.

While the environments abstract away many real-world microstructure features (e.g., queue priority, adverse selection), they provide a controlled and theoretically grounded setting for isolating the effects of inventory risk, execution uncertainty, and price dynamics on market-making strategies.

3.2 Agents

This section describes the decision-making entities used in the simulation study. All agents—analytical, heuristic, and learning-based—are designed to solve the same sequential decision

problem: choosing bid and ask quotes at each time step based on the observed market state. To ensure a fair comparison, all agents interact with the environment through a common abstract interface and are evaluated under identical market conditions.

3.2.1 Unified Agent Interface (Abstract Agent Class)

All agents in this study conform to a common abstract agent interface. Conceptually, an agent implements a policy

$$\pi: \mathcal{S} \to \mathcal{A},$$

mapping the observed state $s_t \in \mathcal{S}$ to a quoting action $a_t \in \mathcal{A}$ at each discrete time step.

The only mandatory operation required of every agent is the ability to compute an action given the current observation. This design choice reflects the fact that, from the environment's perspective, analytical formulas, heuristic rules, and learned policies are indistinguishable once they produce bid–ask quotes.

Training, parameter updates, and persistence are treated as optional capabilities. Analytical and heuristic agents implement fixed policies and therefore do not require training, while reinforcement learning agents extend this interface with learning and model-saving functionality.

This abstraction serves two methodological purposes. First, it enforces comparability: all agents face the same information set and control the same action space. Second, it enables clean separation between market dynamics and decision rules, ensuring that performance differences arise from the agent's policy rather than implementation artifacts.

3.2.2 Reinforcement Learning Agents

Reinforcement learning agents approach the market-making problem as a sequential decision-making task under uncertainty. Unlike analytical models, which rely on explicit assumptions about price dynamics and order arrival processes, reinforcement learning agents learn their quoting policies directly through interaction with the simulated market.

Formally, the environment is modeled as a Markov Decision Process (MDP) defined by the tuple

$$(\mathcal{S}, \mathcal{A}, P, r),$$

where $\mathcal{S}$ is the state space described in Section 3.1.5, $\mathcal{A}$ is the continuous action space of spread and skew parameters, P is the transition kernel induced by price dynamics and order executions, and r is the risk-adjusted reward function defined in Section 3.1.7.

Reinforcement learning agents seek to learn a policy π_θ that maximizes the expected cumulative reward

$$\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right],$$

where γ is the discount factor. In this study, γ is typically set close to one, reflecting the market maker's focus on cumulative profit over a finite trading horizon rather than myopic gains.

The reinforcement learning framework is particularly well suited to market making for three reasons.
First, it naturally accommodates inventory risk, which enters the reward function in a nonlinear manner.
Second, it allows policies to adapt to non-stationary and partially observable environments, such as regime-switching volatility and jump diffusion.
Third, it does not require specifying parametric models for price dynamics or execution probabilities, beyond what is embedded in the simulator.

Multiple reinforcement learning agents are considered in this study. All share the same state and action representations but differ in policy architecture and learning dynamics. The first and simplest reinforcement learning agent used as a baseline is a Proximal Policy Optimization agent.

Summary of Reinforcement Learning Agents

The reinforcement learning agents considered in this study span a range of algorithmic paradigms and architectural choices. Together, they form a structured family of learning-based market-making strategies that differ along four key dimensions: learning paradigm, policy stochasticity, representational capacity, and temporal modeling. This diversity allows for a systematic examination of *why* and *when* reinforcement learning may outperform classical stochastic control approaches.

At the most basic level, the **PPO agent** serves as a memoryless, on-policy baseline. It learns a stochastic quoting policy using policy-gradient updates constrained by a clipped objective, ensuring stable learning in the presence of noisy reward signals. From a financial perspective, this agent tests whether a generic policy-gradient method, operating on a low-dimensional state representation, can recover inventory-aware quoting behavior similar to that prescribed by analytical models such as Avellaneda–Stoikov. Its performance in environments that closely match classical assumptions provides a reference point for evaluating learning-based approaches.

The **Deep PPO agent** extends this baseline by increasing the depth of the policy network while keeping the learning algorithm unchanged. This agent isolates the role of *function approximation capacity*. By allowing the policy to represent more complex nonlinear mappings from state variables to quote parameters, Deep PPO assesses whether richer expressiveness alone improves market-making performance as the environment departs from idealized assumptions. Importantly, any performance gains relative to standard PPO can be attributed to representational power rather than changes in learning dynamics.

The **LSTM-based PPO agent** introduces explicit temporal memory into the decision process. By conditioning actions on both the current state and an internal recurrent hidden state, this agent relaxes the effective Markov assumption and allows the policy to internalize information from past observations. In market-making environments with latent volatility regimes, jump risk, or other forms of partial observability, this capability is particularly valuable. Economically, the LSTM agent can be interpreted as learning an implicit filter over unobserved market conditions, adjusting spreads and inventory skew based on inferred regime dynamics rather than instantaneous observations alone.

In contrast to the on-policy PPO family, the **Soft Actor–Critic (SAC) agent** represents an off-policy approach grounded in maximum-entropy reinforcement learning. By explicitly encouraging stochasticity in the policy, SAC balances exploitation of profitable quoting strategies with continued exploration. This feature is especially relevant in market-making environments characterized by non-stationarity and sudden structural changes. The use of experience replay also makes SAC substantially more sample efficient, enabling it to reuse historical market trajectories when learning value estimates and policies.

The **Deep SAC agent** (named LSTM SAC in the codebase, but technically is a Deep SAC model that tries to include the temporal information in the decision making process through a non-recurrent architecture) builds on this framework by increasing network depth, thereby enhancing the expressiveness of both the policy and value functions. While this agent does not incorporate explicit temporal memory, its high-capacity function approximators may capture complex nonlinear interactions between inventory, price dynamics, and time remaining. As such, it provides a useful intermediate benchmark for assessing whether increased representational power can substitute for explicit recurrence in partially observable environments.

Finally, the **TD3 agent** represents a deterministic, off-policy control paradigm. By learning a deterministic mapping from states to actions, TD3 emphasizes precision and stability in quote placement. Its use of clipped double Q-learning and delayed policy updates improves robustness relative to earlier deterministic methods. From a market-making standpoint, TD3 can be viewed as learning a disciplined quoting rule that reacts consistently to observed states, making it a natural contrast to the explicitly stochastic policies learned by PPO and SAC.

Taken together, these agents form a comprehensive reinforcement learning benchmark set. They allow the analysis to disentangle the effects of on-policy versus off-policy learning, stochastic versus deterministic policies, shallow versus deep architectures, and memoryless versus recurrent decision rules. This structured comparison is central to the project objective: identifying the specific market conditions under which learning-based market-making strategies offer meaningful advantages over classical stochastic control models, and clarifying the sources of those advantages when they arise.

3.2.2.1 Proximal Policy Optimization (PPO) Agent

Proximal Policy Optimization (PPO) is a policy-gradient method designed to achieve stable learning in continuous action spaces. It improves upon vanilla policy-gradient approaches by constraining policy updates, preventing excessively large deviations that can destabilize training.

In the context of market making, the PPO agent learns a parameterized stochastic policy

$$\pi_\theta(a_t \mid s_t),$$

where the action a_t consists of the spread and skew control variables defined in Section 3.1.6. The policy is represented by a neural network that maps the observed market state—time remaining, normalized price information, recent price changes, and inventory level—to quoting decisions.

The PPO objective maximizes a clipped surrogate function of the form

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_{t(\theta)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right],$$

where $r_t(\theta)$ is the likelihood ratio between the new and old policies and $\hat{A}_t$ is an estimator of the advantage function. This formulation stabilizes learning by limiting how much the policy can change in response to noisy reward signals—a critical property in financial environments with high variance.

The PPO agent operates directly on the low-dimensional state representation introduced earlier, without access to full order book information. As a result, any inventory control or quote-skewing behavior must be learned implicitly from the reward structure rather than engineered explicitly. This makes PPO a natural learning-based analogue to classical inventory-based market-making models.

During evaluation, the learned policy is executed deterministically using the mean action of the policy distribution. This ensures that reported results reflect the structural behavior of the learned strategy rather than stochastic exploration noise.

The PPO agent serves as a baseline reinforcement learning benchmark in this study. Its performance in environments that closely match the assumptions of the Avellaneda–Stoikov model provides a reference point for assessing whether learning-based methods can recover analytically optimal behavior. Its behavior in more complex environments—such as those with jumps or regime switches—illustrates the potential advantages and limitations of model-free learning relative to classical stochastic control.

3.2.2.2 Deep Proximal Policy Optimization (Deep PPO) Agent

While the standard PPO agent provides a strong baseline for continuous control problems, its representational capacity may be insufficient in environments where the optimal quoting policy depends on complex nonlinear interactions between price dynamics, inventory, and time remaining. To address this limitation, a Deep PPO agent is introduced, which extends the PPO framework by employing a deeper neural network architecture.

From a functional approximation perspective, the Deep PPO agent replaces the shallow policy network with a multi-layer nonlinear function

$$\pi_\theta(a_t \mid s_t) = f_\theta(s_t),$$

where f_θ consists of multiple hidden layers with nonlinear activation functions. This increased depth allows the policy to approximate more complex mappings from the state space to quoting decisions, enabling richer inventory management and spread-skew trade-offs.

The motivation for this agent is closely tied to the progressive enrichment of the simulation environment. As the market dynamics depart from the idealized assumptions of the Avellaneda–Stoikov framework—through mean reversion, jump diffusion, or regime-switching volatility—the optimal policy may no longer be well-approximated by simple linear or shallow nonlinear functions. In such cases, deeper architectures may capture higher-order interactions, such as nonlinear inventory asymmetries or state-dependent aggressiveness.

Importantly, the Deep PPO agent does not alter the underlying reinforcement learning objective or optimization procedure. It uses the same clipped policy-gradient formulation as standard PPO, ensuring that any observed performance differences can be attributed to representational capacity rather than changes in the learning algorithm itself. As such, Deep PPO serves as a controlled experiment on the role of function approximation depth in market-making reinforcement learning.

3.2.2.3 Recurrent PPO Agent with LSTM Memory

A fundamental limitation of feedforward reinforcement learning policies is their reliance on the Markov property: all relevant information about the future must be contained in the current observed state. In market-making environments with latent regimes, volatility clustering, or jump dynamics, this assumption is often violated. Even when the environment is Markovian in theory, partial observability can induce effective non-Markovian behavior from the agent's perspective.

To address this issue, a recurrent reinforcement learning agent based on Long Short-Term Memory (LSTM) networks is introduced. This agent uses a recurrent variant of PPO, in which the policy is augmented with an internal memory state that evolves over time:

$$(h_t, a_t) = \pi_\theta(s_t, h_{t-1}),$$

where h_t denotes the latent hidden state of the LSTM.

The inclusion of memory allows the agent to condition its quoting decisions not only on the current observation but also on a compressed summary of past states. This is particularly relevant in environments with regime-switching volatility, where the current volatility state is not directly observed, or in jump diffusion settings, where recent price behavior carries predictive information about short-term risk.

From a financial interpretation standpoint, the LSTM agent can be viewed as learning an implicit filter over latent market conditions. Rather than explicitly estimating volatility regimes or jump intensities, the agent internalizes such information through its recurrent state and adjusts spreads and inventory skew accordingly.

The recurrent PPO agent operates under the same reward structure and action space as the feedforward agents, ensuring comparability. However, its policy class is strictly richer, as it can implement history-dependent strategies that are unattainable for memoryless policies. This makes the LSTM agent particularly well suited for assessing whether temporal dependencies and partial observability materially affect optimal market-making performance.

During evaluation, the recurrent hidden state is reset at the beginning of each episode to ensure independence across simulated trading days. This prevents information leakage across episodes and preserves the episodic interpretation of the trading horizon.

3.2.2.4 Soft Actor–Critic (SAC) Agent

While PPO-based agents rely on on-policy updates, an alternative class of reinforcement learning methods is based on **off-policy actor–critic algorithms**. Among these, Soft Actor–Critic (SAC) is particularly well suited for continuous control problems such as market making.

SAC differs fundamentally from PPO in its objective. Rather than maximizing expected cumulative reward alone, SAC optimizes a **maximum-entropy objective** of the form

$$\mathbb{E}\left[\sum_{t=0}^{T} r_t + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\right],$$

where $\mathcal{H}$ denotes the policy entropy and α is a temperature parameter controlling the exploration–exploitation trade-off. This formulation explicitly encourages stochasticity in the learned policy, allowing the agent to explore a wider range of quoting behaviors.

From a market-making perspective, entropy regularization has a natural interpretation. In uncertain or rapidly changing market conditions, maintaining a degree of randomness in quote placement can prevent premature convergence to overly aggressive or brittle strategies. This is particularly relevant in environments with jump risk or stochastic volatility, where deterministic policies may overfit to transient patterns.

The SAC agent learns both:

- a stochastic policy $\pi_\theta(a_t \mid s_t)$, and
- a pair of Q-functions Q_{ϕ_1}, Q_{ϕ_2},
  which are trained using off-policy updates drawn from a replay buffer.

The use of experience replay makes SAC significantly more sample efficient than on-policy methods such as PPO. This allows the agent to reuse past market trajectories, which is advantageous in environments with long episodes and relatively expensive simulations.

In this study, SAC serves as a benchmark for evaluating whether off-policy learning and entropy maximization provide tangible benefits over policy-gradient approaches in the market-making setting.

3.2.2.5 Deep SAC Agent (High-Capacity Approximation)

Due to the lack of a recurrent SAC implementation in standard reinforcement learning libraries, a true LSTM-based SAC agent is not considered. Instead, a high-capacity Deep SAC agent is employed as a proxy, using deeper neural network architectures to partially capture complex state interactions. While this approach does not provide explicit temporal memory, it allows the policy and value functions to represent richer nonlinear dependencies in the observed state.

Although, some of the benefits of temporal abstraction can be partially recovered through increased representational capacity. To this end, a Deep SAC agent is introduced, which augments the standard SAC framework with deeper neural network architectures.

Formally, the policy and value functions are parameterized by high-capacity nonlinear function approximators:

$$\pi_\theta(a_t \mid s_t), Q_{\phi_i}(s_t, a_t), \qquad i = 1,2,$$

with deeper multilayer architectures than those used in the baseline SAC agent.

The motivation for this design is twofold.
First, deeper networks may capture nonlinear interactions between inventory, price changes, and time remaining that are difficult to represent with shallow architectures.
Second, in environments with latent regimes or jump dynamics, deep function approximators may implicitly encode longer-term dependencies through complex state transformations, even in the absence of explicit recurrence.

It is important to emphasize that this agent does not possess true memory in the sense of recurrent neural networks. Rather, it tests whether capacity alone, combined with SAC's entropy-driven exploration, is sufficient to approximate history-dependent behavior.

As such, the Deep SAC agent occupies an intermediate position between memoryless feedforward policies and fully recurrent architectures. Its performance helps disentangle the effects of policy expressiveness from those of explicit temporal memory.

3.2.2.6 Twin Delayed Deep Deterministic Policy Gradient (TD3) Agent

The final reinforcement learning agent considered in this study is based on Twin Delayed Deep Deterministic Policy Gradient (TD3), a deterministic off-policy actor–critic algorithm designed to address instability issues in earlier deterministic methods such as DDPG.

Unlike PPO and SAC, TD3 learns a deterministic policy

$$a_t = \mu_\theta(s_t),$$

which maps states directly to actions without sampling from a distribution. This makes TD3 particularly attractive for market-making tasks, where precise and stable quote placement may be desirable.

TD3 improves stability through three key mechanisms:

1. **Clipped double Q-learning**, which reduces overestimation bias by maintaining two critic networks.
2. **Delayed policy updates**, ensuring that the value estimates are sufficiently accurate before updating the policy.
3. **Target policy smoothing**, which adds noise to target actions during critic updates to improve robustness.

From a financial interpretation standpoint, TD3 can be viewed as learning a **highly disciplined quoting rule**, reacting deterministically to market states while maintaining robustness through conservative value estimation. This contrasts with SAC's explicitly stochastic behavior and PPO's reliance on on-policy exploration.

Because TD3 is off-policy, it is also sample efficient and benefits from experience replay. However, its deterministic nature makes it potentially more sensitive to model misspecification and non-stationarity, which makes its comparison with SAC particularly informative.

3.2.3 Agents (Stochastic Control–Based Agents)

This section describes the analytical agents used as theory-driven benchmarks in the simulation study. These agents are derived from classical stochastic control formulations of the market-making problem and implement either exact or approximate versions of optimal quoting rules obtained under stylized assumptions.

Unlike reinforcement learning agents, analytical agents do not adapt through experience. Their policies are fully specified ex ante by model parameters such as volatility, risk aversion, and order arrival intensities. As a result, they provide economically interpretable reference strategies whose behavior can be directly traced back to the underlying optimization problem.

All analytical agents operate under the same abstract agent interface as learning-based agents and interact with the environment through the same bounded action space. This ensures that any

performance differences reflect genuine modeling advantages rather than differences in information access or execution mechanics.

3.2.3.1 AS-Inspired Simple Heuristic Agent

The first analytical benchmark is a simplified heuristic agent inspired by the qualitative structure of the Avellaneda–Stoikov solution. This agent does not solve a Hamilton–Jacobi–Bellman (HJB) equation and is not optimal in any formal sense. Instead, it encodes two core economic principles that emerge from classical inventory-based market making:

1. **Time-dependent spread control**
2. **Inventory-induced quote skewing**

In the original Avellaneda–Stoikov model, both the optimal spread and the reservation price depend explicitly on the remaining time to maturity $T - t$. The heuristic agent mirrors this behavior by choosing a spread control that decreases monotonically as normalized time approaches the terminal horizon.

Similarly, the reservation price in the AS model shifts linearly with inventory. The heuristic agent captures this effect through a linear mapping from inventory to a skew factor, producing asymmetric bid and ask quotes that encourage inventory mean reversion.

Formally, let the observed state be

$$s_t = \left(\frac{t}{T},\ \frac{S_t}{S_0},\ \Delta S_t,\ \frac{q_t}{q_{\max}}\right),$$

where q_t is the inventory and $q_{\max}$ is the inventory bound. The agent produces an action

$$a_t = (\text{spread_factor}_t, \text{skew_factor}_t),$$

with

$$\text{spread_factor}_t \propto (T - t), \qquad \text{skew_factor}_t \propto q_t.$$

This agent serves as a low-complexity baseline that isolates the value of *economic structure* without relying on full stochastic control optimality.

3.2.3.2 Avellaneda–Stoikov Closed-Form Agent

The primary analytical benchmark is an agent implementing the closed-form quoting rules derived in Avellaneda and Stoikov (2008). This agent corresponds directly to the solution of a finite-horizon stochastic control problem under exponential utility.

Original Continuous-Time Model (See Appendix: A)

In the Avellaneda–Stoikov framework, the mid-price S_t evolves as

$$dS_t = \sigma\, dW_t,$$

and market order arrivals at distance δ from the mid-price follow Poisson processes with intensities

$$\lambda^{\pm}(\delta) = Ae^{-k\delta}.$$

The market maker maximizes expected exponential utility of terminal wealth:

$$\max_{\{\delta_b(t),\delta_a(t)\}} \mathbb{E}[-\exp\,(-\gamma W_T)],$$

where γ denotes risk aversion.

Solving the associated HJB equation yields approximate closed-form expressions for:

1. **The optimal half-spread**

$$\delta^*(t) = \frac{1}{\gamma}\ln\left(1+\frac{\gamma}{k}\right) + \frac{1}{2}\gamma\sigma^2(T-t),$$

2. **The reservation price**

$$r(q,t) = S_t - \gamma\sigma^2(T-t)q.$$

The reservation price induces an inventory-dependent asymmetry in bid and ask quotes:

$$p_b = r(q,t) - \delta^*(t), p_a = r(q,t) + \delta^*(t).$$

Mapping the AS Solution to the Simulation Environment

A key methodological challenge is embedding these continuous-time, absolute-price formulas into a discrete-time environment with a bounded action space. In the simulation framework, agents do not set p_b and p_a directly. Instead, they output a normalized action

$$a_t = (\text{spread_factor}_t, \text{skew_factor}_t) \in [-1,1]^2,$$

which the environment interprets as:

$$\delta_{\text{env}} = \delta_0 \left(1 + \frac{1}{2}\text{spread_factor}_t\right),$$
$$\text{skew_term}_t = \frac{1}{2}\delta_{\text{env}} \cdot \text{skew_factor}_t,$$
$$p_b = S_t - \delta_{\text{env}} - \text{skew_term}_t, p_a = S_t + \delta_{\text{env}} - \text{skew_term}_t.$$

To embed the AS solution, the analytical agent computes:

$$\tau = T - t, \qquad \delta^* = \delta^*(\tau), \qquad \Delta r = \gamma\sigma^2\tau q,$$

and selects action components such that:

$$\delta_{\text{env}} \approx \delta^*, \text{skew_term}_t \approx \Delta r.$$

This yields the approximate mappings:

$$\text{spread_factor}_t \approx 2\left(\frac{\delta^*}{\delta_0} - 1\right), \text{skew_factor}_t \approx \frac{2\Delta r}{\delta_{\text{env}}}.$$

Clipping is applied to ensure feasibility within the bounded action space, reflecting practical constraints absent from the original continuous-time model.

Interpretation and Scope

This agent represents a close possible implementation of the Avellaneda–Stoikov solution within the constraints of the simulation environment. While discretization, action bounds, and inventory limits introduce approximation error, these constraints are imposed uniformly across all agents.

As a result, the analytical agent provides a principled benchmark for evaluating robustness. In environments that closely satisfy the AS assumptions, it is expected to perform near-optimally. As these assumptions are progressively relaxed—through jumps, mean reversion, or regime-switching volatility—deviations from optimality can be directly attributed to model misspecification rather than implementation artifacts.

This explicit mapping between stochastic control theory and agent-based simulation is central to the project objective: positioning classical analytical market-making models within a broader framework that includes modern learning-based approaches.

3.2.4 Heuristic Baseline Agents

In addition to analytical stochastic control models and reinforcement learning–based agents, this study includes a comprehensive set of heuristic baseline agents. These agents implement simple, rule-based decision strategies that resemble common real-world trading behaviors or stylized models used in market microstructure research. They do not solve an explicit optimization problem and do not adapt through learning. Instead, their behavior is fully specified by fixed rules or simple state-dependent mappings.

The inclusion of heuristic agents serves several methodological purposes. First, they provide economically interpretable reference points against which more sophisticated agents can be evaluated. Second, they allow the analysis to disentangle performance gains arising from complex learning or analytical structure from gains that can already be achieved through straightforward heuristics. Finally, they establish upper and lower behavioral bounds, ranging from purely passive liquidity provision to random, zero-intelligence behavior.

All heuristic agents operate within the same abstract agent interface and interact with the environment through the same normalized action space as other agents. As a result, differences in performance can be attributed to the underlying decision rules rather than differences in market access, information, or execution mechanics.

For clarity and interpretability, heuristic agents are grouped into four conceptual categories, each capturing a distinct dimension of market-making behavior.

Simple heuristic agents represent minimal baseline strategies that either provide liquidity in a static manner or prioritize immediate execution. These agents ignore most state information and serve as lower-bound or sanity-check benchmarks.

Inventory-based heuristic agents explicitly react to the market maker's inventory position but do so using ad hoc rules rather than optimal control. They capture fundamental risk-management intuitions—such as widening spreads or shifting quotes as inventory grows—without relying on volatility estimates, arrival models, or time-to-maturity considerations.

Microstructure-based heuristic agents condition their behavior on short-term price signals. These strategies reflect empirical trading practices in which market makers and liquidity providers adjust quotes in response to recent price movements, momentum, or adverse-selection risk.

Random and zero-intelligence agents remove strategic decision-making altogether. By generating actions randomly from predefined distributions, these agents provide statistical lower bounds on performance and serve as null models for evaluating whether observed profitability exceeds what can be achieved through noise alone.

Together, these four categories form a structured and economically meaningful benchmark set. They ensure that comparisons between analytical models and reinforcement learning agents are

grounded not only in theory but also in realistic and interpretable baselines commonly encountered in practice.

3.2.4.1 Simple Heuristic Agents

3.2.4.1.1 Fixed-Spread Agent

The fixed-spread agent represents the simplest form of passive liquidity provision. It continuously posts symmetric bid and ask quotes at a constant distance from the mid-price, independent of time, inventory, or price dynamics.

Formally, the agent targets a fixed half-spread

$$\delta = m \cdot \delta_0,$$

where δ_0 is the environment's base half-spread and m is a fixed multiplier. The corresponding action is chosen such that the environment-implied spread satisfies

$$\delta_{\text{env}} = \delta_0 \left(1 + \frac{1}{2}\text{spread_factor}\right) \approx \delta,$$

with zero skew:

$$\text{skew_factor} = 0.$$

This agent serves as a **pure liquidity benchmark**. Its performance reflects the profitability of providing constant liquidity without inventory control or signal conditioning.

3.2.4.1.2 Market-Order-Only Agent

The market-order-only agent emulates aggressive execution behavior rather than liquidity provision. Because the environment does not support explicit market orders, this behavior is approximated by collapsing the spread to its minimum and skewing quotes aggressively in the direction that reduces inventory.

Concretely:

- the spread factor is set to its minimum value,
- the skew factor is chosen to push quotes toward immediate execution in the direction of inventory reduction.

This agent represents traders who prioritize **immediate execution over spread capture**, such as liquidation strategies or urgency-driven order flow. Its inclusion allows the analysis to compare liquidity provision against purely aggressive trading behavior.

3.2.4.2 Inventory-Based Heuristic Agents

Inventory-based heuristic agents explicitly condition their quoting behavior on the current inventory level, but do so using simple, ad hoc rules rather than optimal control. These agents reflect practical risk-management heuristics commonly used by liquidity providers to limit inventory exposure.

3.2.4.2.1 Inventory-Based Spread Scaling Agent

The inventory spread-scaling agent increases the quoted spread as the absolute inventory grows, reflecting the intuition that larger positions should be compensated by higher expected spread revenue.

Formally, the agent targets a half-spread

$$\delta(q) = \delta_0(1+\alpha \mid q \mid),$$

where δ_0 is the environment's base half-spread, q is the inventory level, and $\alpha > 0$ controls sensitivity to inventory magnitude.

This target spread is mapped into the environment's action space via

$$\text{spread_factor} \approx 2\left(\frac{\delta(q)}{\delta_0} - 1\right), \text{skew_factor} = 0.$$

Quotes remain symmetric around the mid-price, and inventory control is achieved purely through reduced execution probability as inventory grows. This agent isolates the effect of spread-based risk management without directional quote adjustments.

3.2.4.2.2 Inventory-Based Quote Shifting Agent

The inventory quote-shifting agent keeps the spread fixed and instead skews quotes asymmetrically to encourage inventory mean reversion.

The skew component is chosen proportional to inventory:

$$\text{skew_factor}_t \propto q_t,$$

with the spread held constant at its base level.

Within the environment, this results in bid and ask prices shifting away from the mid-price in opposite directions, making executions that reduce inventory more likely. This heuristic closely mirrors the qualitative structure of the reservation price adjustment in the Avellaneda–Stoikov model but omits explicit dependence on volatility, time-to-maturity, or arrival intensities.

Together, the spread-scaling and quote-shifting agents provide complementary inventory-control mechanisms and serve as interpretable baselines for evaluating more sophisticated inventory-aware strategies.

3.2.4.3 Microstructure-Based Heuristic Agents

Microstructure-based agents react to short-term price signals rather than inventory or long-horizon considerations. These strategies are motivated by empirical observations that many market participants condition their behavior on recent price movements.

3.2.4.3.1 Mid-Price Following Agent

The mid-price-following agent conditions its quoting asymmetry on the most recent price change ΔS_t. The agent interprets short-term price movements as directional signals and skews quotes in anticipation of continued movement.

Formally, the agent sets a constant spread and chooses

$$\text{skew_factor}_t \propto \tanh\,(\Delta S_t),$$

where the hyperbolic tangent ensures bounded responses.

A positive price change induces skewing behavior that favors building a long position, while a negative change favors short exposure. This agent resembles trend-following liquidity provision, where quotes track short-term momentum rather than remaining centered on the current mid-price.

3.2.4.3.2 Last-Look (Momentum-Reactive) Agent

The last-look agent also responds to recent price changes but interprets them differently. Instead of building inventory in the direction of the trend, it adjusts quote aggressiveness asymmetrically, effectively reacting to short-term momentum to manage adverse selection.

In practice, the agent skews quotes in the opposite direction of the recent price move:

$$\text{skew_factor}_t \propto -\tanh\,(\Delta S_t).$$

This behavior can be interpreted as a simplified adverse-selection control mechanism: when prices move sharply in one direction, the agent becomes more conservative on the side likely to

be adversely selected. The strategy reflects common “last-look” or defensive quoting practices used by liquidity providers during rapid price movements.

3.2.4.4 Random and Zero-Intelligence Agents

Random and zero-intelligence agents provide lower-bound benchmarks by removing strategic decision-making altogether. These agents are not intended to be profitable; rather, they establish reference distributions for performance that arise purely from randomness.

3.2.4.4.1 Zero-Intelligence Agent (Uniform)

The zero-intelligence agent draws both action components independently from a uniform distribution:

$$\text{spread_factor}_t \sim \mathcal{U}(-1,1), \qquad \text{skew_factor}_t \sim \mathcal{U}(-1,1).$$

Actions are generated independently of the observed market state. This agent corresponds to the classic zero-intelligence traders studied in early market microstructure models, demonstrating that certain aggregate properties can emerge even in the absence of rational behavior.

3.2.4.4.2 Noise Trader (Gaussian)

The Gaussian noise trader generates actions from normal distributions centered at zero:

$$\text{spread_factor}_t \sim \mathcal{N}(0, \sigma^2_{\text{spread}}), \qquad \text{skew_factor}_t \sim \mathcal{N}(0, \sigma^2_{\text{skew}}),$$

with outcomes clipped to the feasible action range.

This agent produces frequent small perturbations around neutral quoting behavior, resembling noisy or partially informed traders whose actions fluctuate randomly but remain concentrated near the mid-price.

3.2.4.4.3 Noise Trader (Uniform-Band)

The uniform-band noise trader restricts random actions to narrower symmetric intervals:

$$\text{spread_factor}_t \sim \mathcal{U}(-r_s, r_s), \text{skew_factor}_t \sim \mathcal{U}(-r_k, r_k),$$

where $r_s, r_k \in (0,1]$.

Compared to the zero-intelligence agent, this variant produces bounded noise around neutral behavior and serves as a smoother stochastic baseline.

3.3 Experimental Design and Experiment Matrix

This section describes the experimental protocol used to evaluate and compare all agents introduced in the previous sections. The objective of the experimental design is to ensure a systematic, fair, and reproducible comparison across different modeling approaches by evaluating each agent under identical market conditions.

3.3.1 General Evaluation Protocol

All experiments follow a unified evaluation pipeline. For a given market environment and agent pair, the agent interacts with the environment over multiple independent episodes (set to 100 by default), each representing a full trading horizon. At the beginning of each episode, the environment is reset and the agent starts with zero inventory and cash. Episodes terminate at the end of the trading horizon or upon truncation imposed by the environment.

For reinforcement learning agents with internal memory (e.g. LSTM-based policies), any recurrent state is explicitly reset at the start of each episode to ensure independence across episodes. This guarantees that performance metrics reflect within-episode decision-making rather than information leakage across simulated trading days.

Agents produce actions at each time step using the same observation space and action space described in Section 3.1.6. After episode termination, terminal profit and loss (PnL) and final inventory are recorded. This process is repeated over a fixed number of evaluation episodes to obtain empirical distributions of outcomes rather than single-path realizations.

3.3.2 Training and Evaluation Separation

For agents that support learning, training and evaluation are treated as distinct phases. Training is performed on a given environment configuration using a fixed number of interaction steps (with reward based early stopping implemented to stop training and restore best weights if the mean reward do not increase for 6 consecutive training episodes equivalent to 60,000 timesteps, by default). After training is complete, the agent is evaluated on freshly initialized environments with independent random seeds.

Analytical agents and heuristic baseline agents do not require training and are evaluated directly. This unified treatment ensures that all agents—learned or non-learned—are ultimately compared using the same evaluation protocol.

3.3.3 Performance Metrics

Evaluation is based on a set of explicitly defined outcome metrics computed from episode-level results. For each agent–environment pair, terminal profit and loss (PnL) is recorded at the end of every evaluation episode, yielding an empirical distribution of returns over repeated simulations.

From this distribution, the following performance measures are reported. First, mean return and return standard deviation are computed to summarize average profitability and

variability. Second, a Sharpe ratio is calculated using the sample mean and standard deviation of terminal PnL, providing a normalized measure of risk-adjusted performance. Third, downside risk is quantified through Value at Risk (VaR) and Expected Shortfall (ES) at both the 95% and 99% confidence levels, capturing tail losses that are particularly relevant for market-making strategies exposed to inventory and price risk.

In addition to PnL-based metrics, inventory-related outcomes are also evaluated. Specifically, the average absolute inventory at episode termination is reported across evaluation runs. This metric provides a direct measure of inventory risk and allows performance differences to be interpreted in terms of the trade-off between profitability and inventory exposure.

All reported metrics are computed from empirical distributions obtained across multiple independent evaluation episodes. This approach ensures that comparisons across agents and environments reflect not only central tendencies but also dispersion and tail behavior, enabling a comprehensive assessment of both return and risk characteristics.

3.3.4 Experiment Matrix

The core structure of the empirical study is an experiment matrix in which each agent is evaluated on each market environment. One dimension of the matrix corresponds to the set of environments described in Section 3.1, ranging from settings that closely match the assumptions of the Avellaneda–Stoikov model to progressively more complex environments with jumps, mean reversion, and regime-switching volatility. The other dimension corresponds to the full set of agents, including analytical agents, heuristic baseline agents, and reinforcement learning agents.

Conceptually, each cell of the matrix represents an independent experiment defined by a specific *(environment, agent)* pair. For each such pair, the agent is evaluated over multiple episodes and a common set of performance metrics is computed. This design enables:

- direct comparison of different agents within the same environment, and
- assessment of how a fixed agent's performance changes as market complexity increases.

By organizing experiments in this matrix form, the analysis can explicitly identify the regimes in which classical stochastic control performs well, the scenarios in which simple heuristics suffice, and the conditions under which reinforcement learning agents deliver measurable improvements.

3.3.5 Reproducibility and Result Storage

All experiments are executed using a unified and standardized experiment runner that manages environment instantiation, agent initialization, optional training, and evaluation under a consistent protocol. This design ensures that each agent–environment pair is evaluated under identical conditions, eliminating implementation-induced variability.

Experimental outputs are stored in a structured directory hierarchy indexed by environment and agent identifiers. For each experiment, both summary metrics and raw episode-level data are

preserved, enabling post hoc analysis and replication of reported results. For learning-based agents, trained model parameters and associated metadata—including environment configurations, agent hyperparameters, and training duration—are optionally saved to facilitate exact reproducibility of learned policies.

To further support transparency and reproducibility, the full source codebase used in this study, including environment implementations, agent definitions, experiment scripts, and evaluation utilities, is publicly available at:

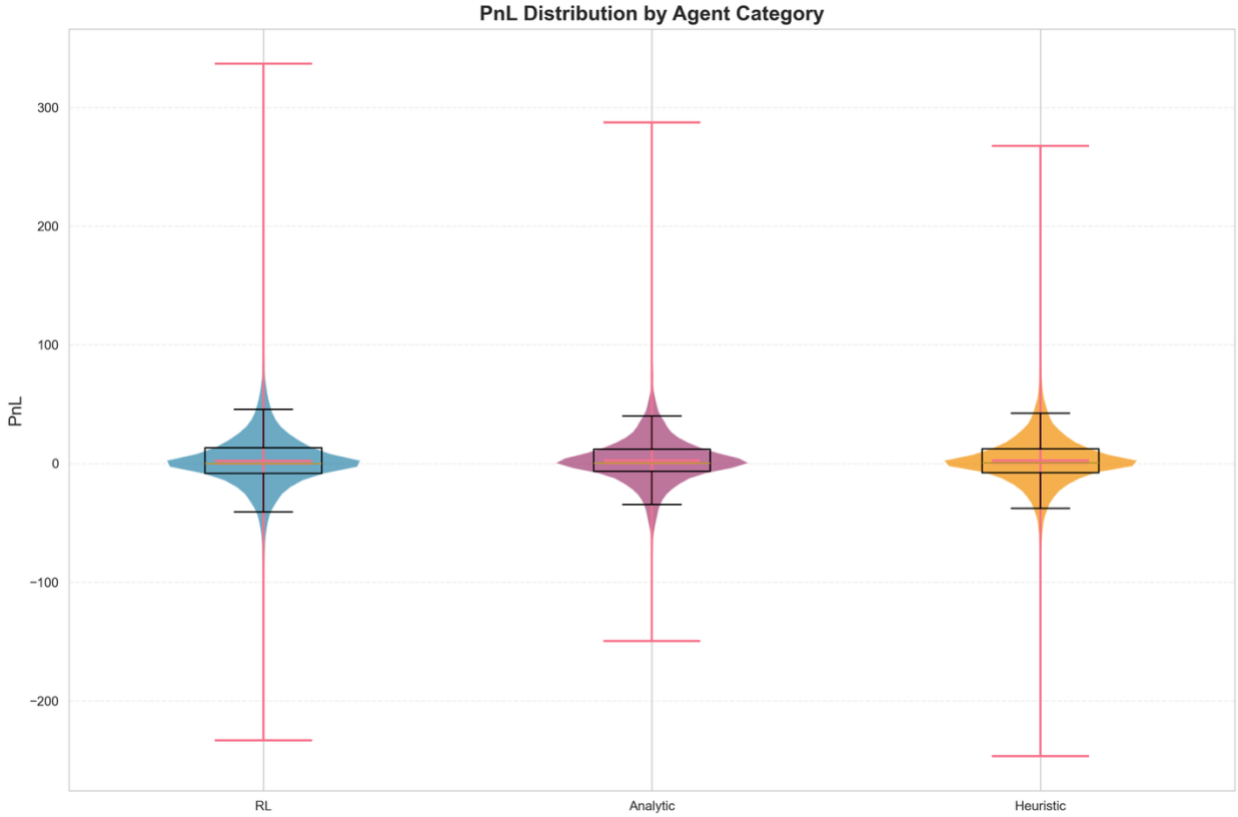
https://github.com/cemyural00/market_making_rl

This repository allows all experiments presented in the report to be independently reproduced and extended, ensuring that the empirical findings reported in subsequent sections are grounded in a fully auditable and reproducible computational framework.

4. Results (See Appendix: E for the comprehensive results data)

This section reports empirical performance across the full experiment matrix (12 environments × 17 agents). Results are presented at two levels: (i) environment-level comparisons to identify which methods dominate under specific market dynamics, and (ii) category-level aggregates (RL vs Analytic vs Heuristic) to summarize systematic differences in profitability, risk, and inventory behavior.

4.1 Cross-Category Summary

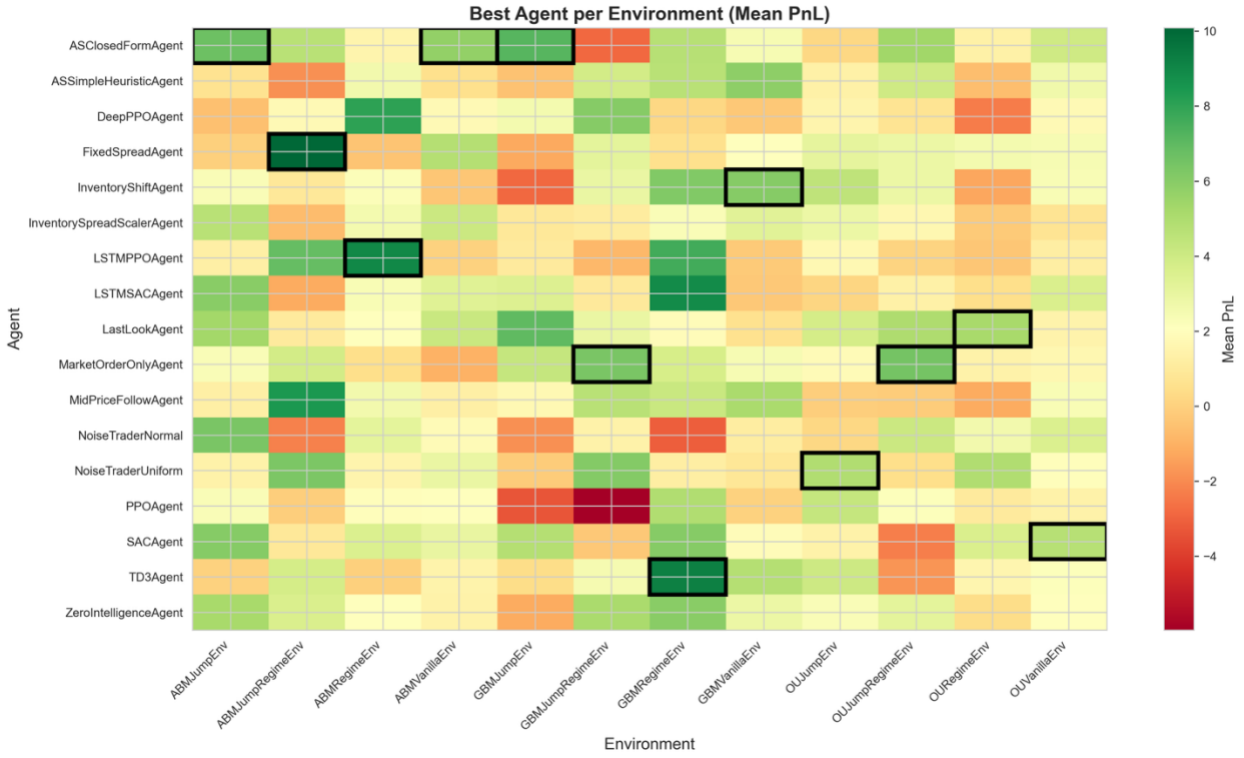


Across all environments, analytical agents achieved the strongest risk-adjusted profile, while RL agents exhibited higher dispersion and materially worse tail risk.

- **Mean PnL (category average):** Analytic **2.67** > Heuristic **2.46** > RL **2.08**
- **Volatility (Std):** Analytic **21.65** < Heuristic **23.31** < RL **26.18**
- **Sharpe:** Analytic **0.132** > Heuristic **0.108** > RL **0.081**
- **Tail risk (VaR/ES):** RL is consistently worse (more negative) than Analytic and typically worse than Heuristic
    - VaR(95): RL **-35.69** vs Analytic **-29.29**
    - VaR(99): RL **-66.41** vs Analytic **-49.51**
    - ES(99): RL **-92.25** vs Analytic **-65.39**
- **Average terminal inventory:** RL **1.34** > Heuristic **1.12** > Analytic **1.05**

The category averages already suggest a clear pattern: RL does not dominate in this simulation suite on a risk-adjusted basis, and its higher mean PnL in some regimes is typically accompanied by meaningfully worse downside risk and larger inventory footprints.

4.2 Profitability by Environment



A consistent finding is that the identity of the best-performing approach changes with market dynamics, and in several complex regimes, simple heuristics match or exceed both RL and analytical baselines in mean PnL.

4.2.1 ABM family (closest to AS assumptions, then enriched with jumps/regimes)

ABMVanillaEnv: The ASClosedFormAgent is the clear benchmark winner in mean PnL (5.76) and also posts a high Sharpe (0.344). RL is positive on average (1.95) but does not challenge the analytical benchmark; heuristics remain competitive (e.g., FixedSpread 4.79).

ABMJumpEnv: The ASClosedFormAgent remains strong (6.64) and dominates average RL (2.51) and average heuristic (3.19) performance. RL's best members (SAC 6.03, LSTMSAC 6.00) approach the AS benchmark on mean, but do not improve on its risk profile.

ABMRegimeEnv: RL becomes highly competitive in mean PnL (RL Avg 4.14) driven by DeepPPO (8.04) and LSTMPPO (8.97); the analytical agents are modest (Analytic Avg 2.07). This is one of the clearest regimes where learning-based agents extract material value, plausibly by adapting spread/skew to regime-driven volatility changes using the low-dimensional state.

ABMJumpRegimeEnv: Mean performance becomes heterogeneous. A key result is that a simple heuristic (FixedSpread) achieves the highest mean PnL (10.09) and strong Sharpe (0.313), with MidPriceFollow also strong (8.45). RL is mixed (RL Avg 2.01) and includes failures (e.g., LSTMSAC mean -1.17). This environment is a first indication that policy learning is not automatically robust to simultaneous jumps + regime switching, and that the simulator structure can still reward simple quoting rules.

Takeaway (ABM block): In the environment most aligned with Avellaneda–Stoikov assumptions, the closed-form strategy is difficult to beat. As complexity is introduced, RL can generate large mean gains in select regimes (ABMRegime), but robustness is not guaranteed, and heuristics can dominate in the most complex ABM variant (ABMJumpRegime).

4.2.2 GBM family (scale effects and multiplicative dynamics)

GBMVanillaEnv: Analytical performance is driven by the ASSimpleHeuristicAgent (5.83) rather than ASClosedForm (2.38). The strongest single performer is a heuristic (InventoryShift 6.04). RL is weak on average (0.99) despite TD3 posting a positive mean (4.81). This indicates that the closed-form AS mapping is less aligned with GBM dynamics under the environment's bounded action parametrization.

GBMJumpEnv: The ASClosedFormAgent again leads (7.16) with LastLook (6.98) close behind. RL average is positive but smaller (1.47) and is dragged down by PPO's negative mean (-3.39).

GBMRegimeEnv: This is the strongest RL environment in the dataset: RL Avg 6.15, with TD3 (9.27) and LSTMSAC (8.88) leading. Here, RL clearly outperforms both analytical (4.69) and heuristic (2.54) averages in mean PnL.

GBMJumpRegimeEnv: RL average is near zero (0.42) with extreme dispersion across agents; meanwhile, heuristics dominate mean PnL (MarketOrderOnly 6.34, NoiseUniform 6.14). The

analytical category is near-flat on average (0.51) because ASClosedForm is negative (-2.86) while ASSimple remains positive (3.88).

Takeaway (GBM block): RL can dominate *mean* outcomes in GBMRegime, but breaks down in GBMJumpRegime, where rule-based aggressiveness (MarketOrderOnly / constrained-random heuristics) captures more PnL under this execution model.

4.2.3 OU family (mean reversion + jump/regime enrichments)

OUVanillaEnv: RL performs well (RL Avg 2.48) and produces the best single mean (SAC 4.74), while ASClosedForm is competitive (3.96). Heuristics are moderate.

OUJumpEnv: Heuristics lead (NoiseUniform 4.92, InventoryShift 4.46) and RL is positive (2.18) but does not dominate.

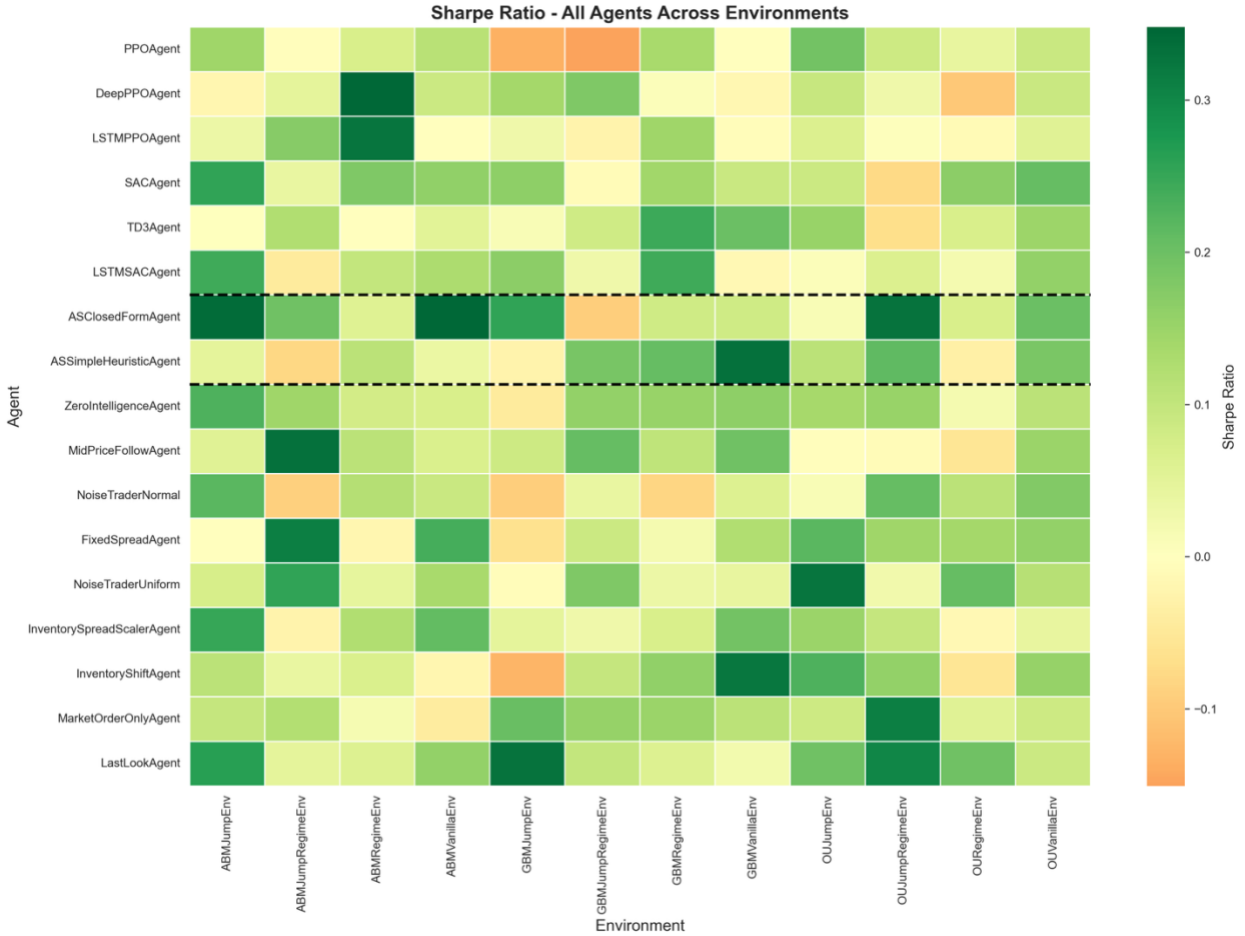
OURegimeEnv: Heuristics again dominate (LastLook 5.17, NoiseUniform 4.92) while RL is modest (0.65) and analytical performance is near-flat (0.37) due to ASSimple being negative.

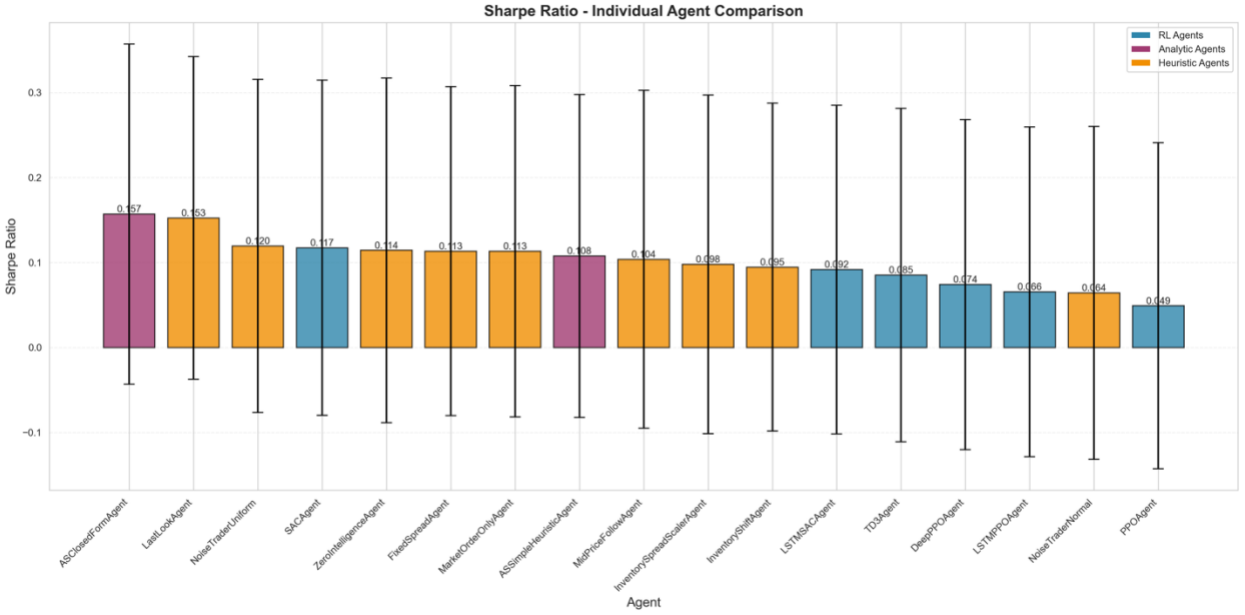
OUJumpRegimeEnv: The best mean is MarketOrderOnly (6.51); analytical agents are also strong (ASClosedForm 5.37). RL collapses on average (0.05) with negative means for SAC (-2.31) and TD3 (-1.70).

Takeaway (OU block): In OU regimes that combine partial observability and discontinuities, RL is not consistently advantaged, and tail outcomes become more severe, suggesting that learned spread/skew policies may be brittle when the environment violates effective Markov assumptions under the chosen state vector.

4.3 Risk-adjusted performance and downside risk

4.3.1 Sharpe ratios: analytical strategies are more reliable

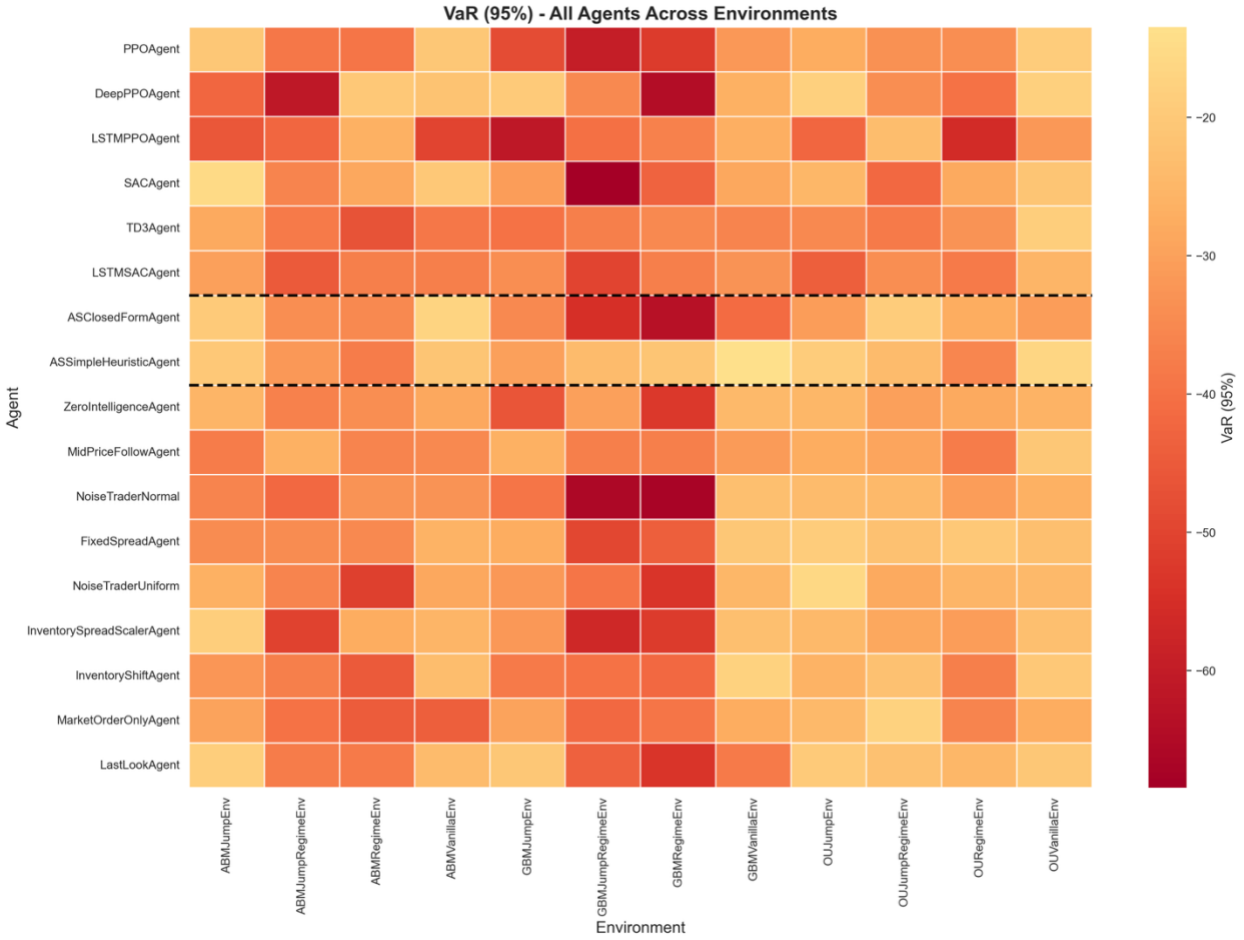


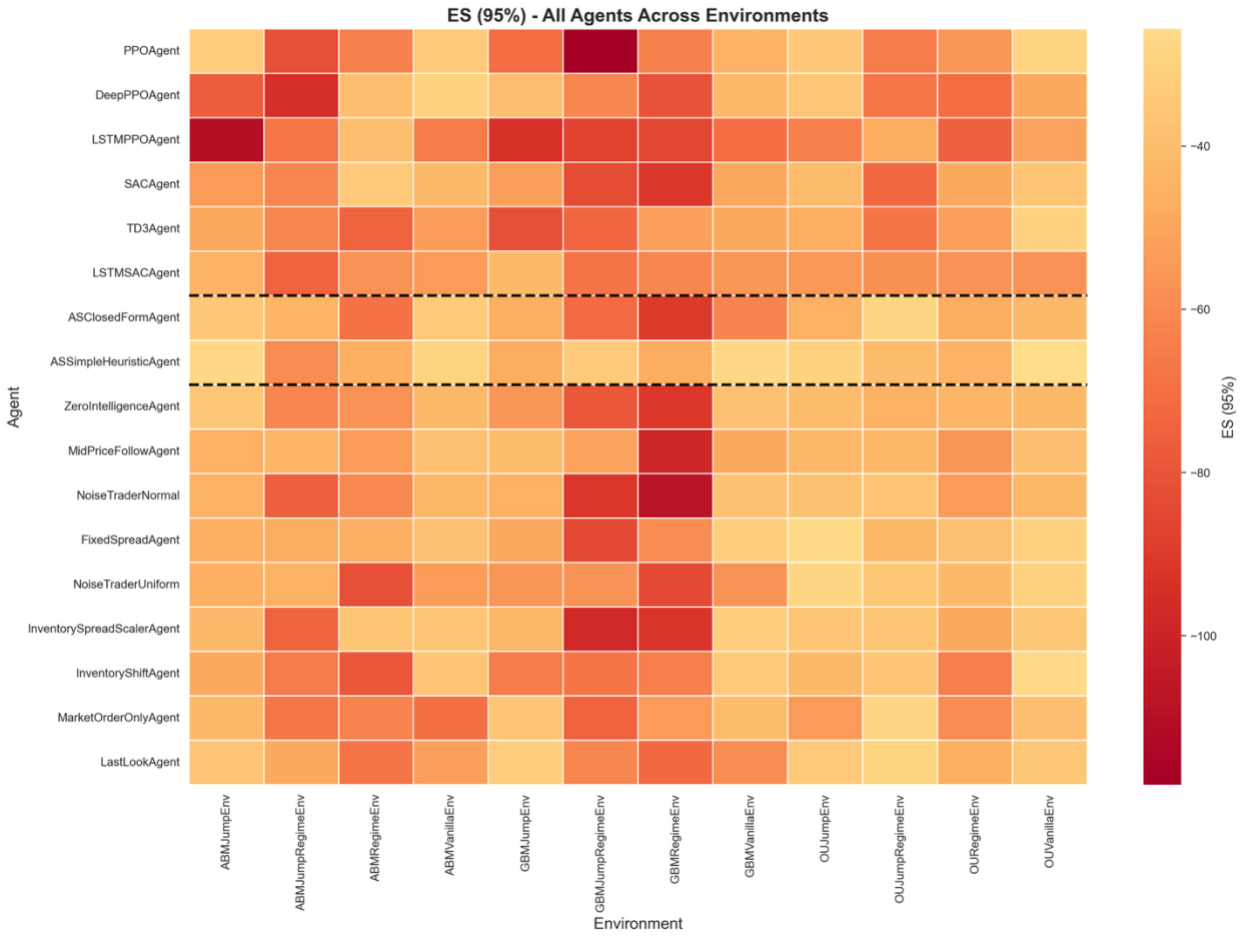


The highest Sharpe values are frequently achieved by ASClosedForm in environments closest to its assumptions (e.g., ABMVanilla 0.344, ABMJump 0.340) and by select heuristics in complex regimes (e.g., ABMJumpRegime FixedSpread 0.313, MidPriceFollow 0.334). RL achieves strong Sharpe only sporadically (e.g., SAC/TD3 in GBMRegime), but the category-level average remains lowest (0.081).

This gap is not primarily driven by mean PnL—since RL sometimes has high means—but by higher return volatility and unstable downside tails.

4.3.2 Tail risk: RL underperforms in VaR/ES almost everywhere





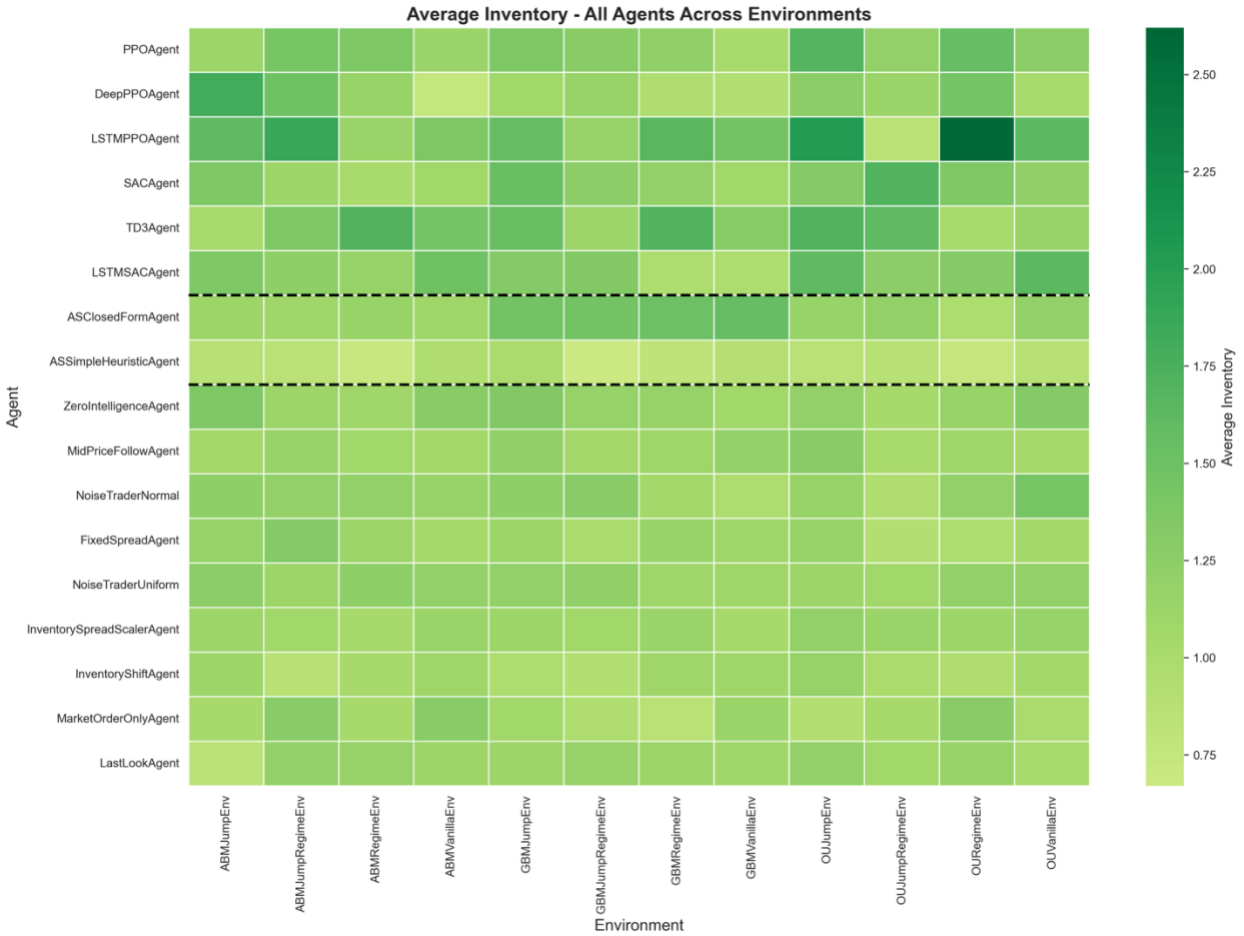
At the category level, RL shows substantially worse left tails:

- VaR(95): RL -35.69 vs Analytic -29.29
- VaR(99): RL -66.41 vs Analytic -49.51
- ES(99): RL -92.25 vs Analytic -65.39

Environment-level tables reinforce this: especially in jump+regime settings, RL's ES(99) becomes extremely negative, indicating that rare inventory/price-path interactions generate disproportionately large losses for learned policies.

A practical implication is that—even when RL yields higher mean PnL in certain regimes (notably GBMRegime)—this comes with worse tail exposure relative to analytical benchmarks.

4.4 Inventory outcomes



Inventory levels are consistently lowest for analytical agents (category average 1.05) and highest for RL (1.34). This is aligned with the tail-risk results: higher inventory footprints amplify exposure to adverse price moves and jumps, particularly when the environment includes regime shifts and discontinuities.

Notably, in OU environments the RL category often holds meaningfully larger terminal inventory (e.g., OURegime RL Avg inventory 1.56 vs Analytic 0.86), coinciding with weaker risk-adjusted results and larger negative ES values. This suggests that learned policies do not reliably internalize inventory liquidation pressure under partial observability, despite the quadratic inventory penalty in the reward.

4.5 Cross-Environment Performance Insights

The results support three high-level conclusions that match the study's "progressive misspecification" design:

1. When the environment matches AS assumptions (ABMVanilla, partially ABMJump), ASClosedForm is a hard benchmark: it delivers both strong mean returns and the best (or near-best) Sharpe with comparatively contained tails.
2. Learning dominates only in selective regimes (most clearly GBMRegime and ABMRegime), where the closed-form mapping is less aligned with dynamics and where RL can exploit nonlinear interactions among time, inventory, and realized price changes.
3. Heuristics remain surprisingly competitive and sometimes dominant in the hardest regimes (especially jump+regime environments), which highlights that (i) the simulator's execution model can reward simple "aggressive" or "defensive" rules, and (ii) RL policies can be brittle when the effective state is only partially observed (volatility regimes, jumps, and mean reversion).

4.5.1 Robustness across environments (win-rates by category)

To summarize robustness, we count which category achieves the highest category-average performance within each environment. Using mean PnL and Sharpe as two complementary criteria, analytical agents lead in 6/12 environments, heuristic agents lead in 4/12, and RL agents lead in 2/12. RL wins are concentrated in regime-only settings (ABMRegime and GBMRegime), whereas the most complex environments that combine jumps and regime-switching are more frequently won by simple heuristics or analytical baselines. This pattern indicates that RL improvements in mean performance are real but regime-specific, and do not translate into consistent dominance across the broader stress-test suite.

5. Conclusion (See Appendix: E)

This report evaluated classical stochastic-control market-making strategies, namely the AS model, against reinforcement learning (RL) agents and rule-based heuristics in a unified experimental framework (12 environments × 17 agents). The study was intentionally structured as a progressive misspecification test: starting from environments closely aligned with Avellaneda–Stoikov (ABM + exponential Poisson fills), then systematically introducing departures that are common in real markets—multiplicative price dynamics (GBM)**,** mean reversion (OU)**,** latent volatility regimes, and jump discontinuities. The goal was to identify where each approach is robust and why performance differences emerge under controlled changes in market dynamics.

5.1 Core findings

(1) Analytical strategies are strongest when their assumptions are only weakly violated.
As expected, analytic agents—especially the AS closed-form implementation—perform best in environments that remain close to the model's structural assumptions (vanilla ABM). In these settings, the analytical policy is effectively "pre-optimized" for the environment: it already encodes the correct economic trade-off between spread capture and inventory risk.

(2) Analytical performance degrades under GBM, OU, latent volatility regimes, and jumps.
As the environment departs from diffusion-only ABM, closed-form policies become increasingly misspecified:

- Under GBM, price risk scales with level, and the distribution of price changes becomes more state-dependent. This changes optimal spread-setting and inventory control in ways a diffusion-calibrated closed form does not naturally internalize under a fixed mapping.
- Under OU, the price process contains a predictable pull toward a long-run mean. Inventory risk and quote placement interact differently because the "cost" of holding inventory depends on where price is relative to the mean and how fast it reverts.
- Under latent volatility regimes, the optimal control becomes regime-dependent; a static mapping that assumes one volatility level will systematically miscalibrate spread and skew when volatility switches.
- Under jumps, the diffusion approximation breaks: inventory risk becomes dominated by rare discontinuities, and strategies that look attractive on average can accumulate large tail losses.

(3) RL was expected to close the gap in those misspecified regimes, but it did so only partially—mainly in volatility regime-change environments. Across the suite, RL's clearest advantages appear in regime-switching volatility environments (ABMRegime and especially GBMRegime), where learning can adapt spread/skew to realized path features that proxy the latent volatility state. Outside this subset—particularly in jump + regime settings—RL generally fails to deliver consistent dominance and often exhibits inferior tail risk.

(4) Simple heuristics remain competitive across regimes—and sometimes dominate in the hardest settings. Rule-based policies (inventory shift, last-look style defensiveness, market-order-only style aggressiveness, fixed spread variants) are frequently competitive and in several complex environments outperform both analytic and RL agents in mean PnL. This is a meaningful finding: it suggests that under this execution model, a large fraction of performance can be captured by relatively simple, robust decision rules that respond directly to inventory and short-horizon price moves.

(5) Off-policy RL generally outperformed on-policy RL in this study. When we compare the RL sub-families directly, SAC / TD3 / Deep-SAC variants tend to be more reliable than PPO / Deep-PPO / LSTM-PPO across environments. Empirically, their advantage appears most clearly in environments with regime changes where sample efficiency and stable critic learning are valuable. On average, the off-policy group shows higher mean performance and better Sharpe than the on-policy group. This is consistent with the fact that off-policy methods benefit from experience replay and can learn from a broader distribution of transitions, which matters in market-making settings where rewards are noisy and the rare paths often drive risk.

5.2 Why did it play out this way?

The results are consistent with three interacting drivers: (i) market dynamics, (ii) the execution model, and (iii) learning stability under a constrained interface.

5.2.1 Market dynamics and misspecification channels

Analytic strategies know what to do when the world looks like ABM.
The AS closed-form strategy is derived precisely for a driftless diffusion with exponentially

decaying fill intensities. In ABM-like environments, it has the right structure: spreads widen with risk and time, and skew pushes inventory back toward zero.

GBM and OU change the meaning of risk and the value of inventory.

In GBM, the same absolute inventory becomes more dangerous as prices rise (risk scales with price level). A quoting rule calibrated to constant absolute volatility may not widen enough at higher levels, or may place skew in a way that fails to align with multiplicative risk.

In OU, the presence of mean reversion changes the effective inventory cost: holding inventory is less risky when the process is expected to revert toward a mean that benefits your position, and more risky when it is expected to revert against you. Without explicit state features capturing distance-to-mean and reversion strength (beyond what is indirectly inferable), both analytic and RL policies can be systematically off.

Latent volatility regimes favor adaptivity, but only if the agent can infer the regime.
Regime-switching volatility is the cleanest case where RL should have an edge: the policy must adapt to a latent state. But with a low-dimensional state vector, that advantage depends on whether the agent can infer the regime using available proxies (e.g., recent price changes). This appears to be exactly where RL shines (regime-only environments) and where it struggles (jump+regime, where inference is confounded by discontinuities).

Jumps are primarily a tail-risk problem.
Jump risk makes rare paths disproportionately important. Policies that maximize mean PnL can still be dominated by occasional large drawdowns driven by inventory × jump interactions. The results reflect this: RL can be positive on average but still show materially worse VaR/ES in jump-rich settings.

5.2.2 Execution model effects (Poisson intensity with exponential decay)

The environment's order execution engine is structured as independent Poisson arrivals with intensities $\lambda(\delta) = Ae^{-k\delta}$. This has two consequences that help explain the observed competitiveness of heuristics:

Profitability is strongly governed by the spread–fill trade-off controlled by A and k.
If intensity is high enough (large A) and decay is not too steep (small k), even simple quoting rules can earn positive spread capture frequently. Conversely, small parameter shifts can meaningfully change which policies look optimal.

The model rewards "structural" behaviors that are easy to hard-code.
Inventory-reducing skew and defensive widening after large price changes are "cheap wins" in this framework because fills respond smoothly to quote distance. This can compress the incremental advantage of RL: if a heuristic already captures the correct qualitative response, RL's additional flexibility may not translate into large gains unless the environment introduces a feature that heuristics cannot track (e.g., latent volatility regime).

This also suggests that hyperparameter sweeps over execution parameters (A, k) could materially change the relative ranking—an important direction for future work.

5.2.3 RL stability and why off-policy methods did better

Market making is a difficult RL task because rewards are noisy, feedback is delayed (inventory risk accumulates), and rare events dominate tail outcomes. Two points matter here:

On-policy methods (PPO family) can be variance-sensitive in long episodes and may require careful tuning (clip range, entropy, GAE parameters, learning rate schedules, normalization) to converge to stable policies.

Off-policy methods (SAC/TD3) reuse experience and can be more sample efficient, which is especially helpful when the environment contains regimes: replay buffers provide mixed-regime transitions that improve learning of state-value structure, even when regime is not directly observed.

The results align with this: off-policy agents more consistently delivered the best RL outcomes, particularly in regime-change environments.

5.3 What could enrich follow-up studies?

This project was designed for interpretability and comparability, but the results naturally point to extensions that could shift RL performance meaningfully.

5.3.1 Hyperparameter and environment sensitivity analysis

A systematic robustness study would vary:

- volatility levels σ (and regime σ_L, σ_H),
- execution parameters A (baseline activity) and k (distance sensitivity),
- jump intensity λ_J and jump size distribution,
- OU parameters κ (reversion speed) and mean level μ,
- inventory penalty η, and inventory bounds $q_{\max}$,
- episode length T and time discretization Δt,
- drift μ in GBM/ABM (set to zero to avoid directional bias by default).

The drift being zero is a strength of the design: it prevents agents from cheating by directionally betting. A follow-up could introduce drift in a balanced way (random sign per episode or regime-dependent drift with zero unconditional mean) to test whether agents develop directional exposure.

5.3.2 Higher dimensional state representations (closing the information gap)

Since RL gains are most plausible when the environment is effectively partially observed (latent regimes), richer state signals could help:

- multi-lag returns and realized volatility estimates,
- rolling measures of volatility or jump indicators,
- execution-related features: time since last fill, imbalance of fills, last action, last spread,
- and eventually real LOB features (depth, imbalance, microprice).

5.3.3 More advanced simulation and real LOB data

Moving toward realism, future work could incorporate:

- alternative execution models (non-Poisson arrivals, Hawkes/self-exciting flow, asymmetric intensities, queue priority, partial fills, variable order sizes, order cancellation, late-filled orders),
- richer microstructure (deeper level LOB structure, adverse selection proxies),
- calibration using historical data, and eventually evaluation on historical limit order book sequences or replay-based simulators.

These changes can materially alter the value of adaptivity: RL often gains relative advantage when execution dynamics are more realistic and less smoothly intensity-driven.

5.3.4 Agent design extensions: reward functions and action spaces

Finally, RL could be strengthened via:

- risk-aware rewards (utility-based, dampened PnL, CvaR (ES)-like penalties, drawdown constraints),
- action-space redesigns (directly choose δ_b, δ_a, allow dynamic side-specific spreads, or discrete quote grids).

5.4 Final interpretation

The study supports a balanced conclusion:

Analytical market making is highly competitive and remains a strong baseline when model assumptions are approximately satisfied.

RL adds value selectively, most clearly when volatility regimes change and the optimal policy must adapt beyond a static volatility calibration.

Heuristics are robust and often surprisingly strong, especially in environments where tail events and regime interactions dominate outcomes.

Within RL, off-policy methods appear better suited to this experiment suite than on-policy PPO variants, likely due to sample efficiency and more stable value learning under noisy returns.

Taken together, these findings suggest that in this simulator class, the path toward RL dominance likely is not more training, but rather better state information, more risk-aware objectives, and

execution models that more closely reflect real microstructure—all while preserving the interpretability and comparability that make analytical strategies valuable in the first place.

6. References

Avellaneda, M., & Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3), 217–224.

Aydoğan, B., Uğur, Ö., & Aksoy, Ü. (2022). Optimal limit order book trading strategies with stochastic volatility in the underlying asset. *Computational Economics*, 62, 289–324.

Carlsson, S., & Regnell, A. (2022). *Reinforcement learning for market making* (Master's thesis). KTH Royal Institute of Technology, Stockholm, Sweden.

Chan, N. T., & Shelton, C. (2001). An electronic market-maker. *Proceedings of the 5th International Conference on Autonomous Agents*. MIT Press.

Ganesh, S., Zheng, H., Vadori, N., Reddy, P., Xu, M., & Veloso, M. (2019). Reinforcement learning for market making in a multi-agent dealer market. *arXiv preprint arXiv:1911.05892*.

Gašperov, B., Begušić, S., Posedel Šimović, P., & Kostanjčar, Z. (2021). Reinforcement learning approaches to optimal market making. *Mathematics*, 9(21), 2689.

Guéant, O., Lehalle, C.-A., & Fernandez-Tapia, J. (2013). Dealing with the inventory risk: A solution to the market making problem. *Mathematics and Financial Economics*, 7(4), 477–507.

Guéant, O. (2017). Optimal market making. *Applied Mathematical Finance*, 24(2), 112–154.

Ho, T. S. Y., & Stoll, H. R. (1981). Optimal dealer pricing under transactions and return uncertainty. *Journal of Financial Economics*, 9(1), 47–73.

Ho, T. S. Y., & Stoll, H. R. (1983). The dynamics of dealer markets under competition. *The Journal of Finance*, 38(4), 1053–1074.

Kim, A. J., Shelton, C. R., Poggio, T., & Lo, A. W. (2002). Market making with reinforcement learning. *Proceedings of the Conference on Computational Intelligence for Financial Engineering*.

Spooner, T., Fearnley, J., Savani, R., & Koukorinis, A. (2018). Market making via reinforcement learning. *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 434–442.

APPENDIX: A — The Avellaneda–Stoikov Optimal Market Making Model

A.1 Introduction

The Avellaneda–Stoikov (2008) model is one of the most influential frameworks for optimal high-frequency market making. It formulates the problem as a **stochastic control problem** in continuous time, where the market maker chooses bid and ask quotes to maximize utility of terminal wealth while controlling inventory risk.

The model provides **closed-form expressions** for optimal bid and ask quotes under assumptions about:

- mid-price dynamics
- order arrival rates
- risk preferences

These explicit formulas form the basis for many academic and industrial market-making systems.

A.2 Model Assumptions

1. Mid-price dynamics: Brownian Motion

The mid-price S_t evolves as:

$$dS_t = \sigma \, dW_t$$

- No drift (market maker is not taking directional bet).
- Volatility σ is constant.

2. Arrival intensities for executions

Limit order executions occur according to **Poisson processes** with intensities:

$$\lambda^{bid}(\delta_b) = Ae^{-k\delta_b}, \lambda^{ask}(\delta_a) = Ae^{-k\delta_a}$$

where:

- $\delta_b = S_t - p^{bid}$ is the distance from mid to bid
- $\delta_a = p^{ask} - S_t$ is the distance from mid to ask

Interpretation:

- Orders close to midprice execute more often.

- Parameter k controls how fast the likelihood decays.

3. Control variables

The market maker chooses bid and ask prices:

$$p_t^{bid}, p_t^{ask}$$

equivalently:

- Half-spread δ_t
- Reservation price r_t (center of quoting)

4. Inventory and wealth dynamics

Let:

- q_t: inventory
- X_t: cash position

Then terminal wealth is:

$$W_T = X_T + q_T S_T.$$

5. Objective function: Exponential utility

The MM maximizes expected utility:

$$\max_{\{p_t^{bid}, p_t^{ask}\}} \mathbb{E}[-\exp(-\gamma W_T)]$$

where $\gamma > 0$ is risk aversion.

As $\gamma \to 0$, the market maker becomes risk-neutral.

A.3 The Hamilton–Jacobi–Bellman (HJB) Equation

Let value function be:

$$u(t, S, q, X) = \max \mathbb{E}[-e^{-\gamma W_T} \mid S_t = S, q_t = q, X_t = X]$$

Because exponential utility is multiplicative, one can separate variables:

$$u(t,S,q,X) = -\exp\left(\quad -\gamma[X + qS + \theta(t,q)]\right),$$

where all optimal control depends on:

- **time left** $T - t$
- **inventory** q

Substituting into the HJB leads to an ODE for $\theta(t,q)$:

$$\partial_t \theta(t,q) = -\frac{1}{2}\gamma\sigma^2 q^2 + \sum_{\pm} \frac{A}{k + \gamma(\pm q)}\left(1 - e^{-\gamma(\theta(t,q\mp 1) - \theta(t,q))}\right)$$

But Avellaneda–Stoikov make further linear approximations to obtain closed-form quotes.

A.4 Reservation Price

After simplifications, the **optimal quoting center** is:

$$r(q,t) = S_t - \frac{\gamma\sigma^2}{2}(T - t)\, q.$$

Interpretation:

- If the MM is **long** (q > 0), reservation price shifts **down**.
- If **short** (q < 0), reservation price shifts **up**.
- The shift increases with time horizon and volatility.

This is the *inventory-aversion skew*.

A.5 Optimal Spread

The optimal **half-spread** is:

$$\delta^{\backslash *}(t) = \frac{1}{\gamma}\ln\left(1 + \frac{\gamma}{k}\right) + \frac{1}{2}\gamma\sigma^2(T - t).$$

Two components:

1. **Execution component:**

$$\frac{1}{\gamma}\ln\left(1+\frac{\gamma}{k}\right)$$

Higher $k \rightarrow$ tighter spreads due to more elastic order flow.

2. **Inventory risk protection:**

$$\frac{1}{2}\gamma\sigma^2(T-t)$$

Higher volatility or longer horizon → wider spreads.

A.6 Optimal Bid and Ask Quotes

Putting everything together:

$$p_t^{bid} = r(q,t) - \delta^{\backslash *}(t)$$
$$p_t^{ask} = r(q,t) + \delta^{\backslash *}(t)$$

This is the **complete optimal quoting strategy**.

Interpretation:

- **Spread widens** when volatility or risk aversion increase
- **Quotes skew** away from the direction of inventory
- **Spread collapses** as $t \rightarrow T$ to reduce terminal inventory risk

A.7 Connection to Project Implementation

Our implementation respects the AS formulas exactly, with a conversion into our environment's action space.

Specifically:

Our env interprets agent actions as:

$$\delta_{\text{env}} = \text{base_delta} \cdot (1 + 0.5 \cdot \text{spread_factor})$$
$$\text{skew_term} = \text{skew_factor} \cdot \frac{\delta_{\text{env}}}{2}$$

So the ASClosedFormAgent solves for:

$$\text{spread_factors.t.}\delta_{\text{env}} \approx \delta^*(t)$$
$$\text{skew_factors.t.skew_term} \approx \gamma\sigma^2(T-t)q.$$

Thus:

- **AS formula is preserved exactly**
- The agent outputs remain within RL-friendly bounds $[-1,1]$
- Quotes in the environment match theoretical AS quotes

This implementation preserves the academic integrity to the AS model while ensuring its practical feasibility.

A.8 Summary Table of Formulas

Concept	Formula	Interpretation
Reservation price	$r(q,t) = S - \frac{\gamma\sigma^2}{2}(T-t)q$	Inventory skew
Half-spread	$\delta^{\backslash *} = \frac{1}{\gamma}\ln\left(1+\frac{\gamma}{k}\right) + \frac{1}{2}\gamma\sigma^2(T-t)$	Execution + risk spread
Bid quote	$S - \frac{\gamma\sigma^2}{2}(T-t)q - \delta^{\backslash *}$	Optimal bid
Ask quote	$S - \frac{\gamma\sigma^2}{2}(T-t)q + \delta^{\backslash *}$	Optimal ask
Objective	$\max\ \mathbb{E}[-e^{-\gamma W_T}]$	Exponential utility

A.9 Why the AS Model Is Foundational in Market Making

- First closed-form dynamic high-frequency quoting model
- Captures both **inventory risk** and **execution risk**
- Elegant convexity and control structure
- Extensible to:
  - drift (signal models)
  - jumps
  - OU mean reversion
  - regime changes
  - multi-asset hedging

APPENDIX: B — Reinforcement Learning Formulation of the Market-Making Problem

This appendix formalizes the market-making problem studied in this project as a reinforcement learning (RL) problem. The purpose is to make explicit the correspondence between the classical stochastic control formulation and the learning-based framework used for empirical evaluation, while avoiding repetition in the main body of the report.

B.1 Markov Decision Process Formulation

The market-making problem is modeled as a finite-horizon, episodic Markov Decision Process (MDP), defined by the tuple

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma),$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $\mathcal{P}$ the transition dynamics, r the reward function, and γ the discount factor.

Each episode corresponds to a single trading horizon $[0, T]$, discretized into time steps of length Δt. The agent acts sequentially by posting bid and ask quotes and receives stochastic rewards driven by order execution and price movements.

B.2 State Space

At each time step t, the agent observes a low-dimensional continuous state vector

$$s_t = \left[\frac{t}{T}, \frac{S_t}{S_0}, S_t - S_{t-\Delta t}, \frac{q_t}{q_{\max}}\right] \in \mathcal{S}.$$

The state components are defined as follows:

- $\frac{t}{T}$: normalized time remaining in the trading horizon,
- $\frac{S_t}{S_0}$: normalized mid-price level,
- $S_t - S_{t-\Delta t}$: most recent price increment,
- $\frac{q_t}{q_{\max}}$: normalized inventory position.

This representation intentionally mirrors the information set assumed in classical market-making models, while remaining sufficiently compact to allow stable learning. The state excludes full limit order book information in order to ensure comparability with analytical stochastic control solutions.

In environments with latent volatility regimes or jump components, the state is only *partially observable*, motivating the use of recurrent policies.

B.3 Action Space and Quote Parameterization

The agent does not act directly on absolute prices. Instead, it selects a continuous action vector

$$a_t = (\alpha_t, \beta_t) \in [-1,1]^2,$$

where:

- α_t controls the **spread width**, and
- β_t controls the **inventory-induced skew**.

These abstract actions are mapped to executable quotes through the environment as follows.

Let δ_0 denote the environment's base half-spread. The effective half-spread is defined as

$$\delta_t = \delta_0 \left(1 + \frac{1}{2}\alpha_t\right),$$

and the skew term is

$$\kappa_t = \frac{1}{2}\beta_t \delta_t.$$

The resulting bid and ask quotes are

$$\begin{aligned} p_t^{\text{bid}} &= S_t - \delta_t - \kappa_t, \\ p_t^{\text{ask}} &= S_t + \delta_t - \kappa_t. \end{aligned}$$

This parameterization enforces realistic quoting behavior, bounds the action space, and allows both analytical and learning-based agents to operate under a unified interface.

B.4 Transition Dynamics

The transition kernel $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ is induced by three stochastic components:

1. **Mid-price dynamics**, governed exogenously by a specified stochastic process (e.g. Brownian motion, Ornstein–Uhlenbeck, jump diffusion, or regime-switching volatility).
2. **Order arrivals**, modeled as independent Poisson processes with intensity functions that decay exponentially in the quote distance from the mid-price.
3. **Inventory and cash updates**, driven by realized executions.

Because order arrivals and price movements are stochastic, transitions are inherently noisy and non-deterministic, consistent with real-world market microstructure.

B.5 Reward Function

The per-step reward is defined as

$$r_t = (W_{t+1} - W_t) - \eta q_{t+1}^2 \Delta t,$$

where:

- $W_t = X_t + q_t S_t$ denotes marked-to-market wealth,
- X_t is cash,
- q_t is inventory, and
- $\eta > 0$ controls inventory risk aversion.

This reward structure approximates the objective of maximizing expected terminal wealth under inventory risk, closely paralleling the exponential utility criterion used in the Avellaneda–Stoikov framework. The quadratic penalty discourages persistent inventory accumulation and stabilizes learning.

B.6 Policy Classes and Objective

A policy $\pi_\theta(a_t \mid s_t)$, parameterized by θ, maps observed states to distributions over actions. Depending on the agent class, policies may be:

- stochastic (e.g. PPO, SAC),
- deterministic (e.g. TD3), or
- recurrent (e.g. LSTM-based PPO).

The learning objective is to maximize expected cumulative reward over an episode:

$$\max_\theta \ \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right],$$

with discount factor γ. In this study, γ is typically set close to one, reflecting the finite-horizon nature of the problem.

B.7 Relation to Stochastic Control

This RL formulation can be viewed as a discretized, model-free analogue of the continuous-time stochastic control problem underlying the Avellaneda–Stoikov model. While the classical approach solves a Hamilton–Jacobi–Bellman equation under strong parametric assumptions, the

RL framework replaces explicit model specification with empirical optimization through interaction.

Importantly, the action parameterization and reward design are chosen to preserve the economic structure of the original problem, enabling meaningful comparisons between analytical, heuristic, and learning-based agents.

APPENDIX: C — Environment Specifications and Parameterization

This appendix documents the market environments used in the experimental analysis. The goal is to provide a transparent and reproducible specification of all stochastic processes, execution mechanisms, and parameter choices employed throughout the report, without burdening the main text with implementation-level detail.

All environments are instantiated using configuration presets defined in a centralized YAML file. This design ensures consistency across experiments and allows all results reported here to be reproduced by loading the corresponding configuration.

C.1 General Environment Structure

All environments share a common structural design. Each environment simulates a single-asset limit order market over a finite trading horizon $[0, T]$, discretized into time steps of length Δt. At each step, the agent posts bid and ask quotes, which may be executed probabilistically based on their distance from the mid-price.

Across all environments, the following components are common:

- **Trading horizon:** $T = 1.0$, interpreted as a normalized trading day.
- **Time discretization:** $\Delta t = 0.0001$, yielding 10,000 steps per episode.
- **Order arrival model:** independent Poisson processes with intensity

$$\lambda(\delta) = Ae^{-k\delta},$$

  where δ is the distance between the quote and the mid-price.

- **Inventory constraint:** inventory is bounded within $[-q_{\max}, q_{\max}]$.
- **Inventory penalty:** quadratic penalty applied through the reward function.

Unless stated otherwise, all environments use identical execution and inventory parameters to isolate the effect of price dynamics.

C.2 Arithmetic Brownian Motion (ABM) Environments

The baseline class of environments models the mid-price S_t as an arithmetic Brownian motion:

$$dS_t = \mu\, dt + \sigma\, dW_t.$$

C.2.1 ABM Vanilla

This environment matches the assumptions of the Avellaneda–Stoikov model most closely.

- $S_0 = 100$
- $\mu = 0$ (drift-free)
- $\sigma = 20$ (absolute volatility)

It serves as the primary benchmark for comparing analytical and learning-based agents under idealized conditions.

C.2.2 ABM with Jump Diffusion

To capture discontinuous price movements, jump diffusion is added:

$$dS_t = \mu\, dt + \sigma\, dW_t + J_t\, dN_t,$$

where N_t is a Poisson process with intensity λ_J, and jump sizes J_t are normally distributed.

This environment tests agent robustness to rare but large price shocks.

C.2.3 ABM with Regime-Switching Volatility

Volatility alternates between low and high regimes according to a discrete-time Markov chain:

$$\sigma_t \in \{\sigma_{\text{low}}, \sigma_{\text{high}}\}.$$

The regime is latent and not directly observed by the agent, introducing partial observability.

C.2.4 ABM with Jumps and Regime Switching

This environment combines jump diffusion and regime-switching volatility, representing the most complex ABM-based setting used in the study.

C.3 Geometric Brownian Motion (GBM) Environments

GBM environments model proportional price changes:

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t.$$

Here, σ represents percentage volatility. These environments are included to test sensitivity to scale-dependent price dynamics.

Vanilla, jump-diffusion, regime-switching, and combined jump–regime variants are implemented analogously to the ABM case, with jumps specified in logarithmic space.

C.4 Ornstein–Uhlenbeck (OU) Environments

Mean-reverting price behavior is modeled using the Ornstein–Uhlenbeck process:

$$dS_t = \kappa(\mu - S_t)\, dt + \sigma\, dW_t.$$

These environments introduce predictable structure into price dynamics, allowing agents to exploit mean reversion while managing inventory risk.

As with ABM and GBM environments, OU variants include jump diffusion, regime-switching volatility, and their combination.

C.5 Execution and Inventory Parameters

Across all environments, execution and inventory-related parameters are held fixed unless otherwise noted:

- Baseline arrival intensity: $A = 5.0$
- Order flow decay parameter: $k = 1.5$
- Base half-spread: $\delta_0 = 1.0$
- Maximum inventory: $q_{\max} = 20$
- Inventory penalty coefficient: $\eta = 0.01$

This standardization ensures that differences in agent performance can be attributed primarily to price dynamics and decision-making strategies rather than execution asymmetries.

C.6 Reproducibility via Configuration Presets

All environment instances used in the experiments are generated from YAML configuration presets. Each preset uniquely defines the stochastic process, execution parameters, and random seed associated with an experiment.

This approach decouples experimental design from code implementation and enables exact replication of all results reported here by loading the corresponding configuration files.

C.7 Parameter Configuration

The following parameters are **identical across all environments** unless stated otherwise:

- Initial mid-price:

$$S_0 = 100$$

- Horizon:

$$T = 1.0$$

- Time step:

$$dt = 0.0001$$

- Baseline order arrival intensity:

$$A = 5.0$$

- Order flow decay (intensity slope):

$$k = 1.5$$

- Base half-spread:

$$\delta_0 = 1.0$$

- Maximum inventory:

$$q_{\max} = 20$$

- Inventory penalty coefficient:

$$\eta = 0.01$$

- Random seed:

$$\text{seed} = 123$$

C.7.1 Arithmetic Brownian Motion (ABM) Environments

ABM Vanilla

$$dS_t = \mu\, dt + \sigma\, dW_t$$

- Drift: $\mu = 0$
- Volatility (absolute): $\sigma = 20.0$

ABM with Jumps

$$dS_t = \mu\, dt + \sigma\, dW_t + J\, dN_t$$

- $\sigma = 20.0$
- Jump intensity: $\lambda_J = 0.1$
- Jump size:

$$J \sim \mathcal{N}(0, 5.0^2)$$

ABM with Regime Switching

Volatility switches between two regimes via a Markov chain:

- Low volatility: $\sigma_L = 10.0$
- High volatility: $\sigma_H = 40.0$
- Transition matrix:

$$\begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix}$$

- Initial regime: low volatility

ABM with Jumps and Regimes

Combination of the above jump diffusion and regime-switching volatility.

C.7.2 Geometric Brownian Motion (GBM) Environments

GBM Vanilla

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

- Drift: $\mu = 0$
- Volatility (percentage): $\sigma = 0.2$ (20% annualized)

GBM with Jumps

Multiplicative (log-normal) jumps:

- Jump intensity: $\lambda_J = 0.1$
- Log-jump mean: 0.0
- Log-jump standard deviation: 0.05

GBM with Regime Switching

- $\sigma_L = 0.1$ (10%)
- $\sigma_H = 0.5$ (50%)
- Same transition matrix and initialization as ABM regimes

GBM with Jumps and Regimes

Combined multiplicative jumps and regime-switching volatility.

C.7.3 Ornstein–Uhlenbeck (OU) Environments

OU Vanilla

$$dS_t = \kappa(\mu - S_t)\, dt + \sigma\, dW_t$$

- Mean-reversion speed: $\kappa = 1.0$
- Long-run mean: $\mu = 100.0$
- Volatility (absolute): $\sigma = 20.0$

OU with Jumps

- Jump intensity: $\lambda_J = 0.1$
- Jump size:

$$J \sim \mathcal{N}(0, 5.0^2)$$

OU with Regime Switching

- $\sigma_L = 10.0$
- $\sigma_H = 40.0$
- Same transition matrix and initialization as ABM regimes

OU with Jumps and Regimes

Combined mean-reversion, jump diffusion, and regime-switching volatility.

C.7.4 Normalization and Agent Interface

- Time is normalized to $[0, 1]$ as t/T
- Inventory is normalized by $q_{\max}$
- Agents observe:

$$o_t = (\text{norm_time},\ S_{\text{norm}}, \Delta S_t, q_{\text{norm}})$$

- Agents act in normalized quote space:

$$a_t = (\text{spread_factor}, \text{skew_factor}) \in [-1,1]^2$$

APPENDIX: D — Agent Specifications and Hyperparameter Configurations

This appendix documents the specification and parameterization of all agents evaluated in the empirical analysis. The purpose is to ensure transparency and reproducibility by clearly describing the hyperparameters, architectural choices, and configuration presets used for reinforcement learning, analytical, and heuristic agents.

All agents are instantiated using centralized YAML configuration files. This design decouples experimental results from implementation details and allows exact replication of all experiments reported here.

D.1 Reinforcement Learning Agents

Reinforcement learning (RL) agents are parameterized through algorithm-specific hyperparameters governing optimization dynamics, policy architecture, and training duration. All RL agents operate in a finite-horizon episodic setting with discount factor $\gamma = 1.0$, reflecting the absence of an infinite-horizon objective.

D.1.1 Proximal Policy Optimization (PPO)

PPO agents serve as the primary on-policy learning baseline. Across configurations, PPO uses generalized advantage estimation (GAE) and clipped policy updates to stabilize training.

Key parameters shared across PPO variants include:

- learning rate 3×10^{-4},
- rollout length $n_{\text{steps}} \in \{512, 1024\}$,
- batch size 128 or 256,
- entropy regularization to encourage exploration.

Three main PPO configurations are used:

- **Basic PPO**, with a standard multilayer perceptron (MLP) policy;
- **Deep PPO**, employing a deeper network with three hidden layers of 256 units to increase representational capacity;
- **Fast PPO**, designed for debugging and rapid experimentation with reduced training horizons.

Early stopping based on evaluation reward is optionally enabled to prevent overtraining and improve computational efficiency.

D.1.2 Recurrent PPO (LSTM-PPO)

To address partial observability and temporal dependencies, recurrent PPO agents are implemented using LSTM-based policies. These agents maintain an internal hidden state that evolves over time, allowing the policy to condition actions on past observations.

LSTM-PPO configurations vary in:

- LSTM hidden state dimension (128 or 256),
- number of recurrent layers (1 or 2),
- post-LSTM MLP depth.

Entropy regularization is typically reduced or disabled for recurrent agents to improve stability. These agents are particularly relevant in environments with regime-switching volatility or jump dynamics.

D.1.3 Soft Actor–Critic (SAC)

SAC agents represent off-policy, entropy-regularized learning methods designed for continuous control. They rely on experience replay buffers and soft target updates, yielding higher sample efficiency relative to on-policy methods.

Key SAC parameters include:

- replay buffer size (100k–500k),
- soft update coefficient $\tau = 0.005$,
- automatic entropy coefficient tuning.

Multiple SAC variants are evaluated to test sensitivity to replay buffer size and training dynamics.

D.1.4 Deep SAC (LSTM-SAC Approximation)

Because recurrent SAC is not available in the chosen framework, temporal expressiveness is approximated through deeper network architectures rather than explicit recurrence. These agents use multi-layer MLPs with up to four hidden layers to capture complex nonlinear patterns.

These configurations serve as a robustness check, isolating the effect of network capacity from explicit memory mechanisms.

D.1.5 Twin Delayed DDPG (TD3)

TD3 agents implement deterministic off-policy control with delayed policy updates and clipped target noise. This class of agents emphasizes precise action selection and stability, providing a contrast to stochastic policy methods such as PPO and SAC.

D.2 Analytical Stochastic Control Agents

Analytical agents are parameterized to match the theoretical assumptions of the Avellaneda–Stoikov framework.

D.2.1 Avellaneda–Stoikov Closed-Form Agent

The closed-form AS agent requires explicit specification of:

- risk aversion parameter γ,
- volatility σ,
- order arrival decay parameter k,
- trading horizon T,
- base spread and inventory bounds.

These parameters are matched exactly to the corresponding environment configuration to ensure theoretical consistency. Separate configurations are provided for arithmetic and geometric Brownian motion environments to account for differences in volatility scaling.

D.2.2 AS-Inspired Heuristic Agent

A simplified AS-style heuristic agent is also included. While not implementing the exact closed-form solution, this agent captures the core economic intuition of inventory-based skewing and time-dependent spread control.

D.3 Heuristic Baseline Agents

Heuristic agents are fully specified by fixed rule parameters and do not involve learning. They are grouped by behavioral class.

D.3.1 Fixed-Spread Agents

These agents quote symmetrically around the mid-price using a constant spread, scaled relative to the environment's base spread. Narrow, standard, and wide spread variants are included.

D.3.2 Inventory-Based Heuristics

Inventory-reactive agents adjust either spread width or quote skew as a function of current inventory. Parameters control the aggressiveness of inventory management, allowing conservative and aggressive variants to be evaluated.

D.3.3 Trend and Microstructure-Based Heuristics

These agents react to short-term price changes by shifting quotes in response to recent price movements. Sensitivity parameters determine how strongly the agent follows momentum or mean-reversion signals.

D.3.4 Market Order and Noise-Based Agents

To establish lower-bound benchmarks, several non-strategic agents are included:

- a market-order-only proxy, implemented via minimal spreads and extreme skewing;
- zero-intelligence agents with uniformly random actions;
- noise traders drawing actions from Gaussian or uniform distributions.

These agents serve as null models against which more structured strategies can be evaluated.

D.4 Configuration Management and Reproducibility

All agent configurations are defined in a centralized YAML file and loaded programmatically at runtime. This approach ensures that:

- hyperparameters are explicitly documented,
- experiments can be exactly reproduced,
- comparisons across agents are not confounded by implicit defaults.

Together with the environment configurations described in Appendix C, these agent specifications fully determine the experimental setup used throughout the report.

Below are the exact parameters used for each agent:

D.4.1.1 PPO Agents (stable-baselines3 PPO)

D.4.1.1.1 ppo_basic

- **Training**
  - total_timesteps: **200000**
  - use_vec_env: **false**
- **Hyperparameters**
  - learning_rate: **0.0003**
  - gamma: **1.0**
  - n_steps: **1024**
  - batch_size: **256**
  - n_epochs: **5**
  - ent_coef: **0.01**
  - vf_coef: **0.5**
  - clip_range: **0.2**
  - gae_lambda: **0.95**
- **Early stopping**
  - enabled: **true**
  - monitor_type: **"reward"**
  - monitor: **"mean_reward"**
  - patience: **12**
  - min_delta: **0.0**
  - mode: **"max"**
  - verbose: **1**
  - eval_freq: **2048**
  - n_eval_episodes: **5**

    - best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.1.2 ppo_deep

- **Training**
    - total_timesteps: **300000**
    - use_vec_env: **false**
- **Hyperparameters**
    - learning_rate: **0.0003**
    - gamma: **1.0**
    - n_steps: **1024**
    - batch_size: **256**
    - n_epochs: **5**
    - ent_coef: **0.01**
    - vf_coef: **0.5**
    - clip_range: **0.2**
    - gae_lambda: **0.95**
- **Policy architecture (policy_kwargs)**
    - net_arch: **[256, 256, 256]**
    - activation_fn: **ReLU**
- **Early stopping** (same structure/values as ppo_basic)
    - enabled: **true**, monitor_type: **"reward"**, monitor: **"mean_reward"**, patience: **12**, min_delta: **0.0**, mode: **"max"**, verbose: **1**, eval_freq: **2048**, n_eval_episodes: **5**, best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.1.3 ppo_fast

- **Training**
    - total_timesteps: **50000**
    - use_vec_env: **false**
- **Hyperparameters**
    - learning_rate: **0.0003**
    - gamma: **1.0**
    - n_steps: **512**
    - batch_size: **128**
    - n_epochs: **5**
    - ent_coef: **0.01**
    - vf_coef: **0.5**
    - clip_range: **0.2**
    - gae_lambda: **0.95**

- **Logging**
  - verbose: **1**
  - tensorboard_log: **null**

D.4.1.1.4 ppo_with_early_stopping

- **Training**
  - total_timesteps: **300000**
  - use_vec_env: **false**
- **Hyperparameters**
  - learning_rate: **0.0003**
  - gamma: **1.0**
  - n_steps: **1024**
  - batch_size: **256**
  - n_epochs: **5**
  - ent_coef: **0.01**
  - vf_coef: **0.5**
  - clip_range: **0.2**
  - gae_lambda: **0.95**
- **Early stopping**
  - enabled: **true**
  - monitor_type: **"reward"**
  - monitor: **"mean_reward"**
  - patience: **12**
  - min_delta: **0.001**
  - mode: **"max"**
  - verbose: **1**
  - eval_freq: **2048**
  - n_eval_episodes: **5**
  - best_model_path: **null**
- **Logging**
  - verbose: **1**
  - tensorboard_log: **null**

D.4.1.2 LSTM-PPO Agents (sb3-contrib RecurrentPPO)

D.4.1.2.1 lstm_ppo

- **Training**
  - total_timesteps: **300000**
  - use_vec_env: **false**
- **Hyperparameters**
  - learning_rate: **0.0003**
  - gamma: **1.0**
  - n_steps: **512**
  - batch_size: **128**
  - n_epochs: **5**

    - ent_coef: **0.0**
    - vf_coef: **0.5**
    - clip_range: **0.2**
    - gae_lambda: **0.95**
- **Recurrent policy (policy_kwargs)**
    - lstm_hidden_size: **128**
    - n_lstm_layers: **1**
    - shared_lstm: **false**
    - net_arch: **[64, 64]**
    - activation_fn: **ReLU**
- **Early stopping**
    - enabled: **true**
    - monitor_type: **"reward"**
    - monitor: **"mean_reward"**
    - patience: **12**
    - min_delta: **0.0**
    - mode: **"max"**
    - verbose: **1**
    - eval_freq: **1024**
    - n_eval_episodes: **5**
    - best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.2.2 lstm_ppo_large

- **Training**
    - total_timesteps: **400000**
    - use_vec_env: **false**
- **Hyperparameters**
    - learning_rate: **0.0003**
    - gamma: **1.0**
    - n_steps: **512**
    - batch_size: **128**
    - n_epochs: **5**
    - ent_coef: **0.0**
    - vf_coef: **0.5**
    - clip_range: **0.2**
    - gae_lambda: **0.95**
- **Recurrent policy (policy_kwargs)**
    - lstm_hidden_size: **256**
    - n_lstm_layers: **2**
    - shared_lstm: **false**
    - net_arch: **[128, 128]**
    - activation_fn: **ReLU**

- **Logging**
  - verbose: **1**
  - tensorboard_log: **null**

D.4.1.3 SAC Agents (stable-baselines3 SAC)

D.4.1.3.1 sac_basic

- **Training**
  - total_timesteps: **300000**
  - use_vec_env: **false**
- **Hyperparameters**
  - learning_rate: **0.0003**
  - gamma: **1.0**
  - buffer_size: **100000**
  - learning_starts: **100**
  - batch_size: **256**
  - tau: **0.005**
  - train_freq: **1**
  - gradient_steps: **1**
  - ent_coef: **"auto"**
- **Early stopping**
  - enabled: **true**
  - monitor_type: **"reward"**
  - monitor: **"mean_reward"**
  - patience: **12**
  - min_delta: **0.0**
  - mode: **"max"**
  - verbose: **1**
  - eval_freq: **2048**
  - n_eval_episodes: **5**
  - best_model_path: **null**
- **Logging**
  - verbose: **1**
  - tensorboard_log: **null**

D.4.1.3.2 sac_large_buffer

- **Training**
  - total_timesteps: **300000**
  - use_vec_env: **false**
- **Hyperparameters**
  - learning_rate: **0.0003**
  - gamma: **1.0**
  - buffer_size: **500000**
  - learning_starts: **1000**
  - batch_size: **256**

    - tau: **0.005**
    - train_freq: **1**
    - gradient_steps: **1**
    - ent_coef: **"auto"**
- **Early stopping** (same structure/values as sac_basic)
    - enabled: **true**, monitor_type: **"reward"**, monitor: **"mean_reward"**, patience: **12**, min_delta: **0.0**, mode: **"max"**, verbose: **1**, eval_freq: **2048**, n_eval_episodes: **5**, best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.4 “LSTM-SAC” Presets (Deep-network SAC workaround)

Note: sb3-contrib does not provide RecurrentSAC; therefore “lstm_sac” refers to SAC with increased network depth to approximate temporal pattern extraction via representational capacity.

D.4.1.4.1 lstm_sac

- **Training**
    - total_timesteps: **300000**
    - use_vec_env: **false**
- **Hyperparameters**
    - learning_rate: **0.0003**
    - gamma: **1.0**
    - buffer_size: **100000**
    - learning_starts: **100**
    - batch_size: **256**
    - tau: **0.005**
    - train_freq: **1**
    - gradient_steps: **1**
    - ent_coef: **"auto"**
- **Policy architecture (policy_kwargs)**
    - net_arch: **[256, 256, 128]**
    - activation_fn: **ReLU**
- **Early stopping**
    - enabled: **true**
    - monitor_type: **"reward"**
    - monitor: **"mean_reward"**
    - patience: **12**
    - min_delta: **0.0**
    - mode: **"max"**
    - verbose: **1**
    - eval_freq: **2048**
    - n_eval_episodes: **5**

    - best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.4.2 lstm_sac_deep

- **Training**
    - total_timesteps: **400000**
    - use_vec_env: **false**
- **Hyperparameters**
    - Same as lstm_sac (learning rate, gamma, buffer size, learning starts, batch size, tau, train_freq, gradient_steps, ent_coef)
- **Policy architecture (policy_kwargs)**
    - net_arch: **[512, 512, 256, 128]**
    - activation_fn: **ReLU**
- **Early stopping** (same structure/values)
    - enabled: **true**, monitor_type: **"reward"**, monitor: **"mean_reward"**, patience: **12**, min_delta: **0.0**, mode: **"max"**, verbose: **1**, eval_freq: **2048**, n_eval_episodes: **5**, best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.1.5 TD3 Agents (stable-baselines3 TD3)

D.4.1.5.1 td3_basic

- **Training**
    - total_timesteps: **300000**
    - use_vec_env: **false**
- **Hyperparameters**
    - learning_rate: **0.0003**
    - gamma: **1.0**
    - buffer_size: **100000**
    - learning_starts: **100**
    - batch_size: **256**
    - tau: **0.005**
    - train_freq: **1**
    - gradient_steps: **1**
    - policy_delay: **2**
    - target_policy_noise: **0.2**
    - target_noise_clip: **0.5**
- **Early stopping**
    - enabled: **true**
    - monitor_type: **"reward"**
    - monitor: **"mean_reward"**

    - patience: **12**
    - min_delta: **0.0**
    - mode: **"max"**
    - verbose: **1**
    - eval_freq: **2048**
    - n_eval_episodes: **5**
    - best_model_path: **null**
- **Logging**
    - verbose: **1**
    - tensorboard_log: **null**

D.4.2 Analytical Agents (Stochastic Control)

D.4.2.1 as_closed_form (ABM/OU-compatible parameters)

- gamma: **0.1**
- sigma: **2.0**
- k: **1.5**
- base_delta: **1.0**
- max_inventory: **20**
- T: **1.0**

D.4.2.2 as_closed_form_gbm (GBM-compatible parameters)

- gamma: **0.1**
- sigma: **0.02**
- k: **1.5**
- base_delta: **1.0**
- max_inventory: **20**
- T: **1.0**

D.4.2.3 as_simple_heuristic

- gamma: **0.1**
- min_spread_factor: **0.2**
- max_spread_factor: **0.8**
- max_inventory: **20**

D.4.3 Heuristic Baseline Agents

D.4.3.1 Fixed Spread

fixed_spread

- fixed_multiplier: **1.0**
- base_delta: **1.0**

fixed_spread_wide

- fixed_multiplier: **1.5**
- base_delta: **1.0**

fixed_spread_narrow

- fixed_multiplier: **0.5**
- base_delta: **1.0**

D.4.3.2 Inventory-Based

inventory_shift

- beta: **0.05**
- max_inventory: **20**

inventory_shift_aggressive

- beta: **0.1**
- max_inventory: **20**

inventory_shift_conservative

- beta: **0.02**
- max_inventory: **20**

inventory_spread_scaler

- alpha: **0.05**
- max_inventory: **20**
- base_delta: **1.0**

D.4.3.3 Trend / Momentum

last_look

- trend_sensitivity: **0.5**

mid_price_follow

- trend_sensitivity: **0.5**
- max_skew: **1.0**

D.4.3.4 Market Order Proxy

market_order_only

- max_inventory: **20**

D.4.3.5 Noise Agents

noise_trader_normal

- spread_std: **0.3**
- skew_std: **0.3**

Note: noise_trader_uniform and zero_intelligence are executed with default parameters (no YAML overrides).

APPENDIX: E — Results and Tables

Please refer to the following links to access the files of the comprehensive results of the study:

EVALUATION_REPORT.md

APPENDIX.md

VISUALIZATION_REPORT.md

RESULTST_AND_CONCLUSIONS.md

AGENT_RISK_RETURN_PROFILES.md

PNL_DISTRIBUTIONS.md